TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

ĐỖ MINH SANG

MSSV: 221651

LỚP: DH22TIN03

TÊN ĐỀ TÀI

XÂY DỰNG HỆ THỐNG CHATBOT HỖ TRỢ TUYỂN SINH ĐẠI HỌC NAM CẦN THƠ (GRAPH RAG)

ĐỒ ÁN CƠ SỞ 02

Ngành: Công Nghệ Thông Tin

Mã số ngành: 7480201

Cần Thơ, ngày… tháng… năm 2026

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN

ĐỖ MINH SANG
MSSV: 221651
LỚP: DH22TIN03

TÊN ĐỀ TÀI

XÂY DỰNG HỆ THỐNG CHATBOT HỖ TRỢ TUYỂN SINH ĐẠI HỌC NAM CẦN THƠ (GRAPH RAG)

ĐỒ ÁN CƠ SỞ 02

Ngành: Công Nghệ Thông Tin
Mã số ngành: 7480201

GIẢNG VIÊN HƯỚNG DẪN

TRẦN VĂN THIỆN

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn chân thành đến quý Thầy, Cô thuộc Khoa Công Nghệ Thông Tin – Trường Đại học Nam Cần Thơ đã luôn tận tụy trong công tác giảng dạy và truyền đạt kiến thức cho sinh viên trong suốt thời gian học tập. Những kiến thức chuyên môn cùng sự hướng dẫn tận tâm của quý Thầy, Cô chính là nền tảng quan trọng giúp chúng em có đủ khả năng nghiên cứu và triển khai đề tài "Xây dựng hệ thống Chatbot hỗ trợ tư vấn tuyển sinh cho Trường Đại học Nam Cần Thơ Graph Rag".

Em xin gửi lời cảm ơn đặc biệt đến Thầy **Trần Văn Thiện**, người đã trực tiếp theo sát và hướng dẫn chúng em trong quá trình thực hiện đề tài. Với sự tận tình hỗ trợ, những góp ý chuyên môn và định hướng rõ ràng từ Thầy, nhóm đã từng bước hoàn thiện hệ thống chatbot nhằm cung cấp thông tin tuyển sinh nhanh chóng và thuận tiện cho người dùng.

Trong quá trình nghiên cứu và thực hiện đề tài "Xây dựng hệ thống Chatbot hỗ trợ tư vấn tuyển sinh cho Trường Đại học Nam Cần Thơ Graph Rag", do kiến thức và kinh nghiệm của nhóm còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được những ý kiến nhận xét và đóng góp từ quý Thầy, Cô để có thể cải thiện và hoàn thiện đề tài tốt hơn.

Cuối lời, chúng em xin kính chúc quý Thầy, Cô Khoa Công Nghệ Thông Tin luôn dồi dào sức khỏe, nhiều niềm vui và đạt được nhiều thành công trong sự nghiệp giảng dạy. Nhóm xin chân thành cảm ơn!

Em xin chân thành cảm ơn!

Cần Thơ, ngày tháng năm 2026

Sinh viên thực hiện

Đỗ Minh Sang

LỜI CAM KẾT

Em xin cam đoan đồ án "Xây Dựng Hệ Thống Chatbot Hỗ Trợ Tuyển Sinh Đại Học Nam Cần Thơ (Graph Rag)" là kết quả nghiên cứu và thực hiện của bản thân em dưới sự hướng dẫn của giảng viên. Toàn bộ nội dung báo cáo, mã nguồn, kết quả thử nghiệm và đánh giá được trình bày trung thực, không sao chép trái phép từ tác phẩm, luận văn hoặc sản phẩm của người khác khi chưa được trích dẫn và ghi nhận đúng quy định.

Nếu có sai phạm về học thuật hoặc bản quyền, em xin hoàn toàn chịu trách nhiệm và chấp nhận các hình thức xử lý theo quy định của nhà trường.

Cần Thơ, ngày tháng năm 2026

Sinh viên thực hiện

Đỗ Minh Sang

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Cần Thơ, ngày .... tháng .... năm 2026

Giảng Viên Hướng Dẫn

Trần Văn Thiện

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU 1
1.1 Đặt vấn đề 1
1.2 Mục tiêu đề tài 2
1.3 Chức năng chatbot 2
1.4 Đối tượng và phạm vi nghiên cứu 3
1.4.1 Đối tượng nghiên cứu 3
1.4.2 Phạm vi nghiên cứu 3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 4
2.1 Nội dung nghiên cứu 4
2.1.1 Cơ sở lý luận 4
2.1.2 Phương pháp nghiên cứu lý luận 4
2.1.3 Môi trường phát triển chatbot 5
2.1.4 Đối tượng và phạm vi nghiên cứu 6
2.2. Cơ sở lý thuyết 6
2.2.1. Khái niệm về đề tài 6
2.2.2. Nguyên lý vận hành hệ thống Chatbot tư vấn tuyển sinh 7
2.2.3. Các chức năng chính 8
2.3 Các công nghệ áp dụng đề tài 9
2.3.1 Môi trường phát triển Visual Studio Code 9
2.3.2 Ngôn ngữ lập trình Python và JavaScript (React) 9
2.3.3 Cơ sở dữ liệu SQLite và Neo4j 9
2.3.4 Kết nối và truy cập dữ liệu (Neo4j Driver, ORM/SQLite) 10
2.3.5 Công cụ mô hình hóa (draw.io) 10
2.3.6 Mô hình phân lớp trong hệ thống web 10
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 11
3.1. Người dùng và quyền hạn 11
3.1.1. Quản trị viên (Admin) 11

3.1.2. Người dùng (User / thí sinh – phụ huynh) 11
3.2. Yêu cầu hệ thống 12
3.2.1 Quản lý người dùng và phân quyền 12
3.2.2 Quản lý dữ liệu tri thức tuyển sinh và Knowledge Graph 12
3.2.3 Tổ chức phiên hội thoại và xử lý câu hỏi 12
3.2.4 Tra cứu và theo dõi tương tác người dùng 13
3.2.5 .Báo cáo và thống kê phục vụ quản trị 13
3.2.6 Tích hợp AI (Gemini) 13
3.3. Thiết kế kiến trúc hệ thống 14
3.3.1 Mục tiêu và nguyên tắc thiết kế 14
3.3.2 Kiến trúc tổng thể (Client – Server) 14
3.3.3 Kiến trúc phân lớp (Layered Architecture) 15
3.3.4 Luồng xử lý nghiệp vụ cốt lõi: Chat tư vấn (Graph RAG) 16
3.3.5 Phân quyền và bảo mật kiến trúc 16
3.3.6 Góc nhìn triển khai (Deployment view) 16
3.4 Rủi ro 17
3.4.1 Rủi ro bảo mật và truy cập trái phép 17
3.4.2 Rủi ro chất lượng và độ tin cậy của câu trả lời AI 17
3.4.3 Rủi ro dữ liệu tri thức (Neo4j) không đồng bộ hoặc seed lỗi 17
3.4.4 Rủi ro hiệu năng và độ trễ (latency) 18
3.4.5 Rủi ro phụ thuộc dịch vụ ngoài (Neo4j cloud, Gemini, email) 18
3.4.6 Rủi ro quyền riêng tư và dữ liệu cá nhân 18
3.5. Mô tả tiến trình 19
3.5.1. Thu thập yêu cầu và khảo sát 19
3.5.2. Phân tích và thiết kế 19
3.5.3. Chuẩn bị dữ liệu tri thức 19
3.5.4. Phát triển ChatBot 20
3.5.5. Kiểm thử 20
3.5.6. Triển khai và vận hành 20

3.5.7. Bảo trì, cập nhật và mở rộng 21
3.5.8. Theo dõi chất lượng và báo cáo kết quả 21
3.6 Mô hình thực thể quan hệ 21
3.6.1 Sơ đồ thực thể quan hệ 21
3.6.2 Mô tả chi tiết các thực thể 22
3.6.3 Mô tả chi tiết các mối quan hệ 22
3.7 Danh sách chức năng và luồng dữ liệu 23
3.8 Mô hình Usecase 26
3.8.1 Sơ đồ Usecase tổng quát 26
3.8.2 Sơ đồ Usecase Admin 27
3.8.3 Mô hình Usecase người dùng (User) 28
3.9 Sơ đồ kiến trúc Graph RAG và mô hình Knowledge Graph Neo4j 29
CHƯƠNG 4: PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU 30
4.1 Mô hình cơ sở dữ liệu 30
4.2 Các bảng cơ sở dữ liệu 30
4.3 Ràng buộc toàn vẹn 32
CHƯƠNG 5: THIẾT KẾ GIAO DIỆN 33
5.1. Giao diện dành cho người dùng (USER) 33
5.1.1 Giao diện màn hình chính người dùng 33
5.1.2 Giao diện đăng ký 34
5.1.3 Giao diện Đăng Nhập. 35
5.1.4 Giao quên mật khẩu 36
5.1.5 Giao diện hồ sơ cá nhân 37
5.1.6 Giao diện khi người dùng đặt câu hỏi 38
5.2. Giao diện dành cho quản trị viên (ADMIN) 39
5.2.1 Giao diện lịch sử chat 39
5.2.2 Giao diện quản lý Knowledge Graph 40
5.2.3 Giao diện thống kê quan tâm & đồ thị Neo4j 41
CHƯƠNG 6: THỬ NGHIỆM VÀ ĐÁNH GIÁ 42

6.1 Kiểm thử 42
6.1.1 Chức năng đăng nhập 42
6.1.2 Chức năng quên mật khẩu 43
6.1.3 Chức năng đăng ký 43
6.1.4 Chức năng chat 44
6.1.5 Chức năng phản hồi 44
6.1.6 Chức năng ngành thêm ngành yêu thích 44
6.1.7 Chức năng phân quyền 45
6.1.8 Chức năng chức năng xây dựng lại đồ thị tri thức Neo4j 45
6.1.9 Chức năng sửa lưu ngành và điểm 45
6.1.10 Chức năng tìm kiếm ngành trong bảng 46
6.2 Đánh giá 46
CHƯƠNG 7: KẾT LUẬN 47

DANH MỤC BẢNG

Bảng 3.1 Danh sách chức năng và nguồn dữ liệu.....23
Bảng 4.1 users(người dùng).....30
Bảng 4.2 chat_sessions(phiên bản hội thoại).....31
Bảng 4.3 chat_messages (tin nhắn) 31
Bảng 4.4 user_favorite_majors(ngành yêu thích).....31
Bảng 4.5 message_feedbacks 32
Bảng 6.1 Kiểm thử chức năng đăng nhập.....42
Bảng 6.2 Kiểm thử quên mật khẩu 43
Bảng 6.3 Kiểm thử chức năng đăng ký 43
Bảng 6.4 Kiểm thử chức năng chat.....44
Bảng 6.5 Kiểm thử chức năng phản hồi 44
Bảng 6.6 Kiểm thử ngành yêu thích 44
Bảng 6.7 Phân quyền 45
Bảng 6.8 Kiểm thử chức năng xây dựng lại đồ thị tri thức Neo4j.....45
Bảng 6.9 Kiểm thử chức năng sửa lưu ngành và điểm.....45
Bảng 6.10 Kiểm thử chức năng tìm kiếm ngành trong bảng.....46

DANH MỤC HÌNH ẢNH

Hình 3.1 Sơ đồ ERD 21
Hình 3.2 Sơ đồ Use Case tổng quát 26
Hình 3.3 Sơ đồ Usecase Admin 27
Hình 3.4 Sơ đồ usecase cho người dùng (User) 28
Hình 3.5 Sơ đồ kiến trúc Graph RAG và mô hình 29
Hình 4.1 Mô hình Database Diagrams 30
Hình 5 1 Giao diện màn hình chính người dùng 33
Hình 5.2 Giao diện đăng ký 34
Hình 5.3 Giao diện đăng nhập 35
Hình 5.4 Giao diện quên mật khẩu 36
Hình 5.5 Giao diện hồ sơ cá nhân 37
Hình 5.6 Giao diện khi người dùng đặt câu hỏi 38
Hình 5.7 Giao diện lịch sử chat 39
Hình 5.8 Giao diện quản lý người dùng 40
Hình 5.9 Giao diện thống kê quan tâm & đồ thị Neo4j 41

CHƯƠNG 1: GIỚI THIỆU

1.1 Đặt vấn đề

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ trong lĩnh vực giáo dục, các trường đại học ngày càng chú trọng ứng dụng công nghệ thông tin nhằm nâng cao hiệu quả trong công tác quản lý và truyền thông tuyển sinh. Đặc biệt, nhu cầu tìm kiếm thông tin của thí sinh và phụ huynh về ngành học, phương thức xét tuyển, học phí, chương trình đào tạo và cơ hội việc làm ngày càng tăng. Vì vậy, việc cung cấp thông tin tuyển sinh một cách nhanh chóng, chính xác và thuận tiện trở thành yêu cầu quan trọng đối với các cơ sở giáo dục đại học.

Thực tế cho thấy, công tác tư vấn tuyển sinh hiện nay tại nhiều trường vẫn chủ yếu dựa vào các kênh truyền thống như website, mạng xã hội, điện thoại hoặc tư vấn trực tiếp. Tuy nhiên, các phương thức này đôi khi chưa đáp ứng kịp thời nhu cầu tra cứu thông tin của thí sinh, đặc biệt trong những giai đoạn cao điểm tuyển sinh khi số lượng câu hỏi tăng cao. Bên cạnh đó, việc tổng hợp, quản lý và cập nhật thông tin tuyển sinh trên nhiều nền tảng khác nhau có thể gây ra sự thiếu nhất quán và mất nhiều thời gian cho đội ngũ quản trị.

Trước thực tế đó, việc xây dựng một hệ thống chatbot hỗ trợ tuyển sinh thông minh là một giải pháp cần thiết nhằm nâng cao hiệu quả tư vấn và cung cấp thông tin cho thí sinh. Hệ thống chatbot có khả năng tự động trả lời các câu hỏi phổ biến liên quan đến thông tin tuyển sinh như ngành đào tạo, điều kiện xét tuyển, học phí, chính sách học bổng, chương trình học và các hoạt động của trường. Đồng thời, việc ứng dụng mô hình Graph RAG (Graph Retrieval-Augmented Generation) giúp hệ thống khai thác dữ liệu theo cấu trúc tri thức, tăng độ chính xác trong việc truy xuất và tạo câu trả lời phù hợp với ngữ cảnh câu hỏi của người dùng.

Đề tài "Xây dựng Chatbot hỗ trợ tuyển sinh Trường Đại học Nam Cần Thơ (Graph RAG)" tập trung vào việc nghiên cứu, phân tích và phát triển một hệ thống chatbot có khả năng tự động hỗ trợ tư vấn tuyển sinh cho thí sinh. Hệ thống hướng đến việc xây dựng một nền tảng thân thiện, dễ sử dụng, có khả năng truy xuất thông tin nhanh chóng và cung cấp câu trả lời chính xác dựa trên cơ sở dữ liệu tuyển sinh của trường. Qua đó, đề tài góp phần nâng cao chất lượng tư vấn tuyển sinh, tối ưu hóa nguồn lực quản lý và thúc đẩy việc ứng dụng trí tuệ nhân tạo trong hoạt động giáo dục.

1.2 Mục tiêu đề tài

Từ nhu cầu thực tế trong hoạt động tư vấn và cung cấp thông tin tuyển sinh cho thí sinh, đề tài hướng đến việc phát triển một hệ thống chatbot hỗ trợ tuyển sinh cho Trường Đại học Nam Cần Thơ ứng dụng mô hình Graph RAG (Graph Retrieval-Augmented Generation). Hệ thống được xây dựng nhằm tự động hóa quá trình giải đáp thắc mắc của thí sinh và phụ huynh, giúp truy xuất thông tin tuyển sinh nhanh chóng, chính xác và nhất quán, đồng thời giảm tải khối lượng công việc cho bộ phận tư vấn tuyển sinh.

Mục tiêu chính của đề tài là thiết kế và triển khai một nền tảng chatbot có khả năng tiếp nhận câu hỏi từ người dùng, phân tích nội dung và đưa ra câu trả lời dựa trên nguồn dữ liệu tuyển sinh của trường. Hệ thống hướng đến việc tích hợp cơ chế truy xuất dữ liệu thông minh thông qua Graph RAG để nâng cao độ chính xác và tính liên kết của thông tin, giúp người dùng dễ dàng tìm hiểu các nội dung như ngành đào tạo, phương thức xét tuyển, học phí, học bổng và các thông tin liên quan khác.

Ngoài ra, đề tài cũng đặt mục tiêu xây dựng hệ thống quản lý dữ liệu tuyển sinh tập trung, hỗ trợ cập nhật và tổ chức thông tin một cách khoa học, đồng thời cung cấp giao diện thân thiện và dễ sử dụng cho người dùng. Thông qua việc triển khai chatbot tư vấn tuyển sinh, hệ thống góp phần nâng cao hiệu quả truyền thông tuyển sinh, cải thiện trải nghiệm tra cứu thông tin của thí sinh và thúc đẩy việc ứng dụng trí tuệ nhân tạo trong lĩnh vực giáo dục.

1.3 Chức năng chatbot

Quản lý người dùng: đăng ký, đăng nhập, quên mật khẩu (gửi email reset), cập nhật thông tin cá nhân, phân quyền Admin/Người dùng. Hỏi đáp tuyển sinh: cung cấp thông tin về ngành học, điểm chuẩn, phương thức xét tuyển, học bổng và nhập học thông qua Chatbot. Chat theo phiên: mỗi cuộc hội thoại được gắn với session_id, lưu trữ và duy trì ngữ cảnh trò chuyện. Xem lịch sử trò chuyện: cho phép người dùng truy xuất lại các nội dung đã trao đổi trước đó. Cập nhật hồ sơ cá nhân: nhập khối thi, điểm dự kiến để phục vụ tư vấn. Tư vấn cá nhân hóa: gợi ý ngành học phù hợp dựa trên thông tin hồ sơ người dùng. Quản lý ngành quan tâm: thêm/xóa danh sách ngành yêu thích. Đánh giá phản hồi: người dùng có thể đánh giá câu trả lời (hữu ích hoặc báo sai) để cải thiện hệ thống. Tương tác nâng cao: hỗ trợ nhập bằng giọng nói (Web Speech API) và nghe phản hồi bằng Text-to-Speech. Quản trị hệ thống: quản lý tài khoản, theo dõi lịch sử chat và dữ liệu vận hành. Quản lý Knowledge Graph: theo dõi, cập nhật và rebuild dữ liệu trên Neo4j. Tích hợp AI (Graph RAG): kết hợp truy xuất dữ liệu từ Knowledge Graph và mô hình AI để tạo câu trả lời chính xác, có ngữ cảnh; hỗ trợ tư vấn tuyển sinh thông minh và trò chuyện tự nhiên. Giao diện thân thiện, dễ sử dụng cho cả thí sinh và quản trị viên.

1.4 Đối tượng và phạm vi nghiên cứu

1.4.1 Đối tượng nghiên cứu

Nghiên cứu quy trình xây dựng và vận hành hệ thống Chatbot hỗ trợ tư vấn tuyển sinh, bao gồm: tiếp nhận và xử lý câu hỏi người dùng, quản lý phiên trò chuyện (session), lưu trữ lịch sử hội thoại và cung cấp thông tin tuyển sinh như ngành học, điểm chuẩn, phương thức xét tuyển, học bổng

Nghiên cứu cơ chế tư vấn cá nhân hóa dựa trên thông tin hồ sơ người dùng (khối thi, điểm dự kiến), quản lý tài khoản, lưu danh sách ngành học quan tâm và thu thập phản hồi đánh giá câu trả lời. Nghiên cứu khả năng tích hợp AI (Gemini API) kết hợp với mô hình Graph RAG và Knowledge Graph (Neo4j) nhằm nâng cao độ chính xác, tính ngữ cảnh của câu trả lời, đồng thời hỗ trợ mở rộng dữ liệu tri thức và cải thiện khả năng hội thoại thông minh của hệ thống. Thời gian thực hiện đề tài: học kỳ 2 năm 4 tại Trường Đại học Nam Cần Thơ.

1.4.2 Phạm vi nghiên cứu

Xây dựng hệ thống Chatbot tư vấn tuyển sinh Đại học Nam Cần Thơ (DNC) trên nền tảng ứng dụng web với Frontend sử dụng React/Vite và Backend sử dụng FastAPI. Ngôn ngữ và thư viện được sử dụng bao gồm JavaScript (React) cho giao diện người dùng và Python (FastAPI) cho dịch vụ phía máy chủ; đồng thời tích hợp mô hình ngôn ngữ lớn (LLM – Gemini) nhằm hỗ trợ sinh câu trả lời tự động. Hệ quản trị cơ sở dữ liệu bao gồm SQLite dùng để lưu trữ dữ liệu người dùng và lịch sử hội thoại, cùng với Neo4j được sử dụng để xây dựng cơ sở tri thức dạng đồ thị (Knowledge Graph) phục vụ mô hình Graph RAG.

Phạm vi chức năng của hệ thống tập trung vào các nghiệp vụ chính như hỏi đáp tuyển sinh (bao gồm thông tin về ngành học, điểm chuẩn, phương thức xét tuyển, học bổng), quản lý tài khoản và hồ sơ cơ bản, lưu trữ lịch sử trò chuyện, quản lý danh sách ngành quan tâm và đánh giá phản hồi câu trả lời, đồng thời hỗ trợ nhập liệu bằng giọng nói và phát âm thanh phản hồi bằng công nghệ Text-to-Speech. Đề tài không đi sâu vào triển khai hệ thống ở quy mô lớn như đa máy chủ hay cân bằng tải, cũng như chưa tích hợp đầy đủ với các hệ thống tuyển sinh nội bộ hoặc bên ngoài; hệ thống chủ yếu đóng vai trò hỗ trợ tư vấn và tra cứu thông tin dựa trên dữ liệu đã được xây dựng trong cơ sở tri thức đồ thị.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Nội dung nghiên cứu

2.1.1 Cơ sở lý luận

Quá trình thực hiện đồ án giúp tôi hiểu rõ hơn về mô hình tổ chức và vận hành của một hệ thống chatbot tư vấn tuyển sinh theo hướng hiện đại, bao gồm các nghiệp vụ chính như tiếp nhận và xử lý câu hỏi của thí sinh, phân loại ý định, truy xuất thông tin tuyển sinh (ngành học, điểm chuẩn, phương thức xét tuyển, học bổng), lưu trữ lịch sử hội thoại, quản lý tài khoản và hồ sơ cơ bản, lưu danh sách ngành học quan tâm và thu thập phản hồi nhằm đánh giá chất lượng câu trả lời. Đây là nền tảng quan trọng để xây dựng một hệ thống chatbot hỗ trợ hiệu quả cho công tác tư vấn tuyển sinh, góp phần nâng cao khả năng tiếp cận thông tin và giảm tải cho bộ phận tư vấn.

Đề tài đồng thời là cơ hội để tôi củng cố và vận dụng các kiến thức đã học như: cơ sở dữ liệu quan hệ (SQLite), cơ sở dữ liệu đồ thị (Neo4j), phân tích và thiết kế hệ thống, xây dựng API với FastAPI (Python), phát triển giao diện người dùng với React/Vite, cũng như xử lý logic nghiệp vụ và kết nối dữ liệu. Bên cạnh đó, việc tích hợp trí tuệ nhân tạo (AI/LLM – Gemini API) kết hợp với mô hình Graph RAG đã mở ra hướng tiếp cận mới trong việc sinh câu trả lời dựa trên ngữ cảnh được truy xuất từ Knowledge Graph, góp phần nâng cao độ chính xác, tính nhất quán và khả năng giải thích nguồn thông tin trong quá trình tư vấn.

Thông qua các giai đoạn khảo sát yêu cầu, thiết kế hệ thống, xây dựng mô hình dữ liệu, triển khai truy xuất tri thức, lập trình, kiểm thử và hoàn thiện sản phẩm, tôi có cơ hội tiếp cận toàn diện quy trình phát triển chatbot trong thực tế. Đồng thời, đồ án cũng giúp tôi nâng cao kỹ năng tư duy hệ thống, giải quyết vấn đề và khả năng thích ứng với các công nghệ mới. Đây là cơ sở quan trọng để tiếp tục phát triển và hoàn thiện hệ thống trong tương lai theo hướng mở rộng nguồn dữ liệu tuyển sinh, cải thiện chất lượng tri thức đồ thị, tối ưu trải nghiệm hội thoại và bổ sung các chức năng quản trị nhằm đáp ứng tốt hơn nhu cầu thực tiễn.

2.1.2 Phương pháp nghiên cứu lý luận

Phương pháp quan sát trực tiếp: khảo sát các website và ứng dụng chatbot tư vấn tuyển sinh cũng như các hệ thống hỏi đáp có tích hợp AI tương tự nhằm tìm hiểu cách tổ chức giao diện hội thoại, quản lý phiên chat, xử lý đăng nhập, lưu trữ lịch sử và thu thập phản hồi người dùng; từ đó phân tích ưu điểm, hạn chế và rút ra mô hình phù hợp cho hệ thống Chatbot tuyển sinh DNC.

Phương pháp thu thập tài liệu: tìm kiếm và tổng hợp tài liệu từ giáo trình, bài báo, internet và tài liệu kỹ thuật liên quan đến FastAPI (Python), React/Vite, SQLite, Neo4j, Graph RAG, vector embedding, vector search, Web Speech API (nhập giọng nói), Text-to-Speech, cùng với các tài liệu tuyển sinh chính thống như ngành học, điểm chuẩn, phương thức xét tuyển, học bổng để xây dựng bộ dữ liệu và cơ sở tri thức.

Phương pháp phân tích và thiết kế hệ thống: từ dữ liệu thu thập được tiến hành phân tích yêu cầu, mô hình hóa chức năng (Use Case), mô hình hóa luồng dữ liệu (DFD), thiết kế kiến trúc tổng thể (Frontend – Backend – LLM – Neo4j – SQLite), thiết kế và chuẩn hóa cơ sở dữ liệu quan hệ (SQLite), đồng thời xây dựng mô hình tri thức và các mối quan hệ trong Neo4j nhằm phục vụ truy xuất ngữ cảnh cho Graph RAG.

Phương pháp chuyên gia: tham khảo ý kiến giảng viên hướng dẫn và những người có chuyên môn trong lĩnh vực tuyển sinh và công nghệ nhằm hiệu chỉnh phạm vi chức năng, chuẩn hóa nội dung tri thức, đánh giá chất lượng câu trả lời và hoàn thiện hệ thống theo yêu cầu thực tế.

2.1.3 Môi trường phát triển chatbot

Hệ thống được phát triển bằng công cụ Visual Studio Code, hỗ trợ lập trình và quản lý mã nguồn hiệu quả. Phần frontend được xây dựng bằng React.js kết hợp Vite và Tailwind CSS nhằm tạo giao diện chat trực quan, thân thiện với người dùng. Phần backend sử dụng Python với FastAPI và Uvicorn để xây dựng API và xử lý các nghiệp vụ của chatbot.

Về cơ sở dữ liệu, hệ thống sử dụng SQLite để quản lý thông tin tài khoản, hồ sơ người dùng, phiên chat, tin nhắn, danh sách ngành quan tâm và phản hồi đánh giá; đồng thời sử dụng Neo4j để lưu trữ dữ liệu dưới dạng đồ thị tri thức phục vụ cho mô hình Graph RAG. Việc kết nối và truy cập dữ liệu được thực hiện thông qua SQLAlchemy đối với SQLite, Neo4j Driver đối với cơ sở dữ liệu đồ thị và REST API để giao tiếp giữa frontend và backend. Ngoài ra, hệ thống tích hợp trí tuệ nhân tạo thông qua Gemini API/LLM để sinh câu trả lời và gợi ý câu hỏi, kết hợp với cơ chế Graph RAG bao gồm các bước như phân loại ý định, tạo embedding và truy xuất ngữ cảnh từ Neo4j nhằm nâng cao độ chính xác và chất lượng phản hồi.

2.1.4 Đối tượng và phạm vi nghiên cứu

Xây dựng các chức năng dành cho thí sinh/người dùng bao gồm: hỏi đáp tư vấn tuyển sinh (ngành học, điểm chuẩn, phương thức xét tuyển, học bổng), đăng ký, đăng nhập, đăng xuất, quên mật khẩu, cập nhật hồ sơ cơ bản (khối thi, điểm dự kiến), xem lịch sử trò chuyện, lưu ngành quan tâm (thêm/xóa), đánh giá câu trả lời (hữu ích hoặc báo sai), đồng thời hỗ trợ nhập liệu bằng giọng nói và nghe đọc câu trả lời bằng công nghệ Text-to-Speech.

Xây dựng các chức năng dành cho quản trị viên (administrator) bao gồm: theo dõi thống kê hệ thống và Knowledge Graph (Neo4j), thực hiện rebuild Knowledge Graph, xem lịch sử chat và quản lý dữ liệu phục vụ quá trình vận hành.

Áp dụng các kỹ thuật và công nghệ triển khai như React kết hợp Vite và Tailwind CSS cho giao diện web; FastAPI (Python) cho backend; SQLite để quản lý tài khoản và lịch sử tương tác; Neo4j để xây dựng cơ sở tri thức dạng đồ thị; đồng thời kết hợp mô hình Graph RAG với các bước như phân loại ý định, tạo embedding khi cần và truy xuất ngữ cảnh từ Neo4j, cùng với việc tích hợp mô hình ngôn ngữ lớn (Gemini API) nhằm sinh câu trả lời dựa trên ngữ cảnh.

Hệ thống được triển khai ở mức ứng dụng hoàn chỉnh trong phạm vi đồ án, đáp ứng nhu cầu tư vấn và tra cứu thông tin tuyển sinh, tuy nhiên chưa tập trung vào triển khai ở quy mô lớn như đa máy chủ hoặc tích hợp toàn diện với các hệ thống tuyển sinh nội bộ.

2.2. Cơ sở lý thuyết

2.2.1. Khái niệm về đề tài

Chatbot tư vấn tuyển sinh là hệ thống có khả năng tiếp nhận câu hỏi của thí sinh, phân tích nhu cầu thông tin và trả lời tự động dựa trên dữ liệu tuyển sinh của nhà trường. Trong bối cảnh tuyển sinh, chatbot tập trung xử lý các nhóm nội dung như thông tin ngành đào tạo, điểm chuẩn, phương thức xét tuyển, tổ hợp môn và học bổng/chính sách hỗ trợ sinh viên. Đây là bài toán có nhiều mảng thông tin liên kết, đòi hỏi hệ thống đảm bảo độ chính xác, tính nhất quán, thời gian phản hồi nhanh và khả năng cập nhật dữ liệu theo từng đợt tuyển sinh.

Hệ thống chatbot tư vấn tuyển sinh là giải pháp công nghệ giúp tự động hóa một phần hoạt động tư vấn, hỗ trợ thí sinh tra cứu thông tin nhanh, giảm tải cho bộ phận tư vấn và nâng cao trải nghiệm người dùng. Hệ thống thường kết hợp các thành phần như giao diện hội thoại, dịch vụ xử lý yêu cầu, lưu trữ lịch sử tương tác và cơ chế quản lý dữ liệu người dùng. Việc áp dụng chatbot giúp hạn chế thao tác thủ công,

giảm sai sót khi cung cấp thông tin, đồng thời tăng khả năng phục vụ đồng thời nhiều người dùng trong mùa tuyển sinh.

Bên cạnh các chức năng cốt lõi, chatbot hiện đại có thể tích hợp mô hình ngôn ngữ lớn (LLM) để sinh câu trả lời tự nhiên và linh hoạt. Tuy nhiên, để hạn chế hiện tượng "bịa thông tin", hệ thống có thể áp dụng Retrieval-Augmented Generation (RAG), đặc biệt là Graph RAG dựa trên đồ thị tri thức (Knowledge Graph). Theo đó, hệ thống truy xuất ngữ cảnh từ cơ sở tri thức (ví dụ Neo4j) về ngành học, điểm chuẩn, phương thức xét tuyển, học bổng… rồi mới đưa vào LLM để sinh câu trả lời, qua đó nâng cao chất lượng tư vấn và tính tin cậy của thông tin.

2.2.2. Nguyên lý vận hành hệ thống Chatbot tư vấn tuyển sinh

Để vận hành hiệu quả hệ thống chatbot tư vấn tuyển sinh, cần tuân thủ các nguyên tắc sau:

Tính bảo mật: bảo vệ dữ liệu tài khoản và thông tin cá nhân của người dùng (đăng ký, đăng nhập, hồ sơ điểm dự kiến và khối thi), đảm bảo an toàn token xác thực, đồng thời phân quyền rõ ràng giữa thí sinh/người dùng và quản trị viên trong các chức năng như thống kê, rebuild knowledge graph và quản lý dữ liệu.

Tính tiện dụng: giao diện hội thoại trực quan, thao tác đơn giản, phản hồi nhanh; hỗ trợ các tiện ích nâng cao trải nghiệm như nhập bằng giọng nói, nghe đọc Text-to-Speech, xem lịch sử chat, lưu ngành quan tâm và đánh giá câu trả lời.

Tính minh bạch và rõ ràng: quá trình xử lý câu hỏi cần có khả năng giải thích theo từng bước như phân loại ý định, truy xuất ngữ cảnh và sinh câu trả lời, đồng thời ghi nhận lịch sử hội thoại và nguồn dữ liệu tham chiếu để người dùng hoặc giảng viên dễ dàng kiểm tra tính hợp lệ.

Tính chính xác và nhất quán: câu trả lời phải dựa trên ngữ cảnh được truy xuất từ cơ sở tri thức Neo4j và dữ liệu tuyển sinh đã được chuẩn hóa, hạn chế suy đoán, đảm bảo thông tin thống nhất giữa các lần hỏi cũng như giữa các nội dung như ngành học, điểm chuẩn, phương thức xét tuyển, tổ hợp môn và học bổng.

Tính tối ưu hóa quy trình: tự động hóa các bước xử lý câu hỏi thông qua mô hình Graph RAG bao gồm phân loại ý định, tạo embedding khi cần, truy vấn Neo4j và tổng hợp ngữ cảnh, kết hợp với mô hình ngôn ngữ lớn để sinh câu trả lời, từ đó giảm thao tác thủ công trong tư vấn, đồng thời thu thập phản hồi từ người dùng nhằm cải thiện chất lượng hệ thống trong tương lai.

2.2.3. Các chức năng chính

Hệ thống Chatbot Tư vấn Tuyển sinh DNC được xây dựng với các chức năng cốt lõi như sau: Hỏi đáp tư vấn tuyển sinh (Chatbot) cho phép thí sinh đặt câu hỏi và nhận câu trả lời tự động về các nội dung như ngành học, điểm chuẩn, phương thức xét tuyển, tổ hợp môn và học bổng; hệ thống hỗ trợ trả lời kèm theo ngữ cảnh truy xuất và gợi ý các câu hỏi liên quan.

Xử lý hội thoại theo phiên và lưu lịch sử bằng cách tạo và quản lý session chat, lưu toàn bộ tin nhắn giữa người dùng và chatbot, đồng thời cho phép xem lại lịch sử trò chuyện.

Quản lý tài khoản người dùng bao gồm đăng ký, đăng nhập, đăng xuất, quản lý thông tin cá nhân và quên mật khẩu thông qua cơ chế gửi email đặt lại mật khẩu bằng reset token; đồng thời phân quyền User và Admin để phục vụ các chức năng quản trị. Cập nhật hồ sơ phục vụ tư vấn cá nhân hóa cho phép người dùng nhập khối thi và điểm dự kiến, từ đó hệ thống đưa ra gợi ý ngành học phù hợp.

Lưu ngành quan tâm (Favorites) hỗ trợ người dùng thêm hoặc xóa danh sách ngành yêu thích nhằm thuận tiện cho việc theo dõi và tra cứu. Đánh giá câu trả lời (Feedback) cung cấp chức năng phản hồi "hữu ích" hoặc "báo sai" để ghi nhận chất lượng phản hồi của chatbot và phục vụ cải tiến hệ thống.

Hỗ trợ đa phương thức nhập xuất bao gồm nhập liệu bằng giọng nói giúp đặt câu hỏi nhanh và chức năng nghe đọc câu trả lời bằng Text-to-Speech nhằm nâng cao trải nghiệm người dùng, đặc biệt trên thiết bị di động.

Quản trị hệ thống (Administrator) cho phép theo dõi thống kê Knowledge Graph trên Neo4j, thực hiện rebuild Knowledge Graph khi cập nhật dữ liệu tuyển sinh hoặc học bổng và xem, kiểm tra dữ liệu lịch sử chat phục vụ vận hành.

Tích hợp trí tuệ nhân tạo thông qua mô hình ngôn ngữ lớn (Gemini API) kết hợp với Graph RAG, trong đó hệ thống thực hiện các bước như phân loại ý định, tạo embedding khi cần, truy xuất ngữ cảnh từ Neo4j và sinh câu trả lời bằng LLM, giúp nâng cao độ chính xác và giảm sai lệch thông tin so với chatbot chỉ sử dụng LLM đơn thuần.

2.3 Các công nghệ áp dụng đề tài

2.3.1 Môi trường phát triển Visual Studio Code

Đề tài sử dụng Visual Studio Code (VS Code) – một môi trường phát triển tích hợp hiện đại – để xây dựng toàn bộ hệ thống web. VS Code hỗ trợ hiệu quả cho việc lập trình Python ở phía backend và JavaScript/React ở phía frontend thông qua các tính năng như quản lý workspace, gợi ý mã nguồn, tích hợp terminal, debug và chạy ứng dụng trong cùng một môi trường. Trong quá trình thực hiện đồ án, VS Code giúp phát triển ứng dụng web bao gồm giao diện chat, chức năng gọi API và quản trị hệ thống một cách trực quan, đồng thời tạo điều kiện thuận lợi khi chỉnh sửa song song giữa frontend và backend. Bên cạnh đó, công cụ còn hỗ trợ tổ chức mã nguồn theo cấu trúc rõ ràng như frontend, backend, data, scripts, góp phần nâng cao khả năng bảo trì và mở rộng hệ thống trong tương lai.

2.3.2 Ngôn ngữ lập trình Python và JavaScript (React)

Python được sử dụng cho backend với framework FastAPI nhằm xây dựng API, xử lý xác thực JWT, triển khai logic hỏi đáp, tích hợp Neo4j và kết nối với Gemini (LLM và embedding). Python có hệ sinh thái thư viện phong phú, phù hợp cho việc xây dựng dịch vụ web và tích hợp trí tuệ nhân tạo.

JavaScript kết hợp với thư viện React (Vite) và Tailwind CSS được sử dụng cho frontend để xây dựng giao diện chat, hiển thị markdown, chức năng đăng nhập, quản lý lịch sử, ngành quan tâm, nhập giọng nói và Text-to-Speech. Mô hình tách biệt client và server giúp giao diện độc lập với xử lý nghiệp vụ, thuận tiện triển khai trên các nền tảng như Hugging Face Space.

2.3.3 Cơ sở dữ liệu SQLite và Neo4j

SQLite (chat_history.db) đóng vai trò là cơ sở dữ liệu quan hệ nhẹ, dùng để lưu trữ thông tin người dùng, phiên chat, tin nhắn, ngành quan tâm và phản hồi đánh giá. SQLite phù hợp cho triển khai đơn giản, không cần cài đặt hệ quản trị cơ sở dữ liệu riêng, thuận tiện cho việc demo và triển khai. Neo4j là cơ sở dữ liệu đồ thị dùng để lưu trữ tri thức tuyển sinh dưới dạng các node như ngành học, điểm chuẩn, tổ hợp môn, phương thức xét tuyển, học bổng và các mối quan hệ như HAS_SCORE, USES_COMBO nhằm phục vụ cho Graph RAG và truy vấn ngữ cảnh có liên kết. Việc kết hợp SQLite cho dữ liệu người dùng và Neo4j cho tri thức đồ thị phù hợp với kiến trúc tổng thể của hệ thống.

2.3.4 Kết nối và truy cập dữ liệu (Neo4j Driver, ORM/SQLite)

Hệ thống sử dụng Neo4j Driver trong Python để thực hiện truy vấn Cypher và cập nhật dữ liệu đồ thị. Đối với SQLite, ứng dụng sử dụng các lớp truy cập dữ liệu theo mô hình DAO hoặc repository để thực hiện các thao tác thêm, sửa, xóa và truy vấn dữ liệu người dùng cũng như lịch sử chat. Cách tổ chức này giúp tách biệt phần truy cập dữ liệu khỏi API và giao diện, làm cho mã nguồn rõ ràng, dễ bảo trì và dễ kiểm soát khi làm việc với nhiều loại cơ sở dữ liệu khác nhau.

2.3.5 Công cụ mô hình hóa (draw.io)

Draw.io được sử dụng để thiết kế các sơ đồ phân tích và thiết kế hệ thống như Use Case, ERD và mô hình Knowledge Graph, giúp mô tả rõ ràng cấu trúc và luồng xử lý trước khi triển khai lập trình. Việc mô hình hóa hỗ trợ quá trình phát triển trở nên nhất quán, giảm sai sót và thuận tiện cho việc trình bày trong báo cáo đồ án.

2.3.6 Mô hình phân lớp trong hệ thống web

Hệ thống được thiết kế theo mô hình phân lớp nhằm tổ chức mã nguồn khoa học, hạn chế phụ thuộc chéo và tăng khả năng bảo trì. Lớp Presentation (Frontend – React) đảm nhiệm giao diện người dùng, bao gồm giao diện chat, form đăng nhập, hiển thị lịch sử và thực hiện gọi API thông qua HTTP. Lớp Application hoặc API (Backend – FastAPI) xử lý nghiệp vụ, xác thực người dùng, điều phối luồng hỏi đáp, phân loại ý định, gọi mô hình ngôn ngữ lớn và kết hợp ngữ cảnh từ Neo4j và SQLite. Lớp Data chịu trách nhiệm truy cập dữ liệu từ SQLite đối với thông tin người dùng và hội thoại, cũng như Neo4j đối với tri thức đồ thị và truy vấn ngữ cảnh, đảm bảo tách biệt hoàn toàn giữa logic nghiệp vụ và chi tiết lưu trữ dữ liệu.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Người dùng và quyền hạn

3.1.1. Quản trị viên (Admin)

Admin là tài khoản đặc biệt trong hệ thống, thường được xác định thông qua quyền quản trị, cho phép truy cập khu vực quản trị trên giao diện web và các API dạng /api/admin mà người dùng thông thường không có quyền sử dụng. Tài khoản này đảm nhiệm việc theo dõi thống kê Knowledge Graph như số lượng node, label và các mối quan hệ nhằm phục vụ giám sát tổng quan hệ tri thức đồ thị. Đồng thời, Admin có thể quản lý và cập nhật dữ liệu tuyển sinh trên Neo4j như xem danh sách ngành, chỉnh sửa thông tin ngành và điểm chuẩn theo chức năng quản trị, cũng như thực hiện rebuild lại đồ thị từ dữ liệu JSON khi cần đồng bộ hoặc khắc phục lỗi dữ liệu. Ngoài ra, hệ thống còn hỗ trợ thống kê hành vi tư vấn nếu được triển khai, chẳng hạn như theo dõi các ngành được tra cứu nhiều nhằm phục vụ mục đích báo cáo và tối ưu hoạt động vận hành.

3.1.2. Người dùng (User / thí sinh – phụ huynh)

Hệ thống hỗ trợ người dùng thực hiện các chức năng như đăng ký, đăng nhập, đăng xuất và cập nhật hồ sơ cá nhân (ví dụ khối thi, điểm dự kiến nếu hệ thống có thu thập). Người dùng có thể sử dụng chức năng quên mật khẩu thông qua luồng đặt lại mật khẩu qua email bằng token hoặc đường dẫn đặt lại mật khẩu tùy theo cấu hình SMTP hoặc dịch vụ gửi mail, không bắt buộc sử dụng OTP nếu thực tế không triển khai. Hệ thống cho phép chat tư vấn tuyển sinh bằng ngôn ngữ tự nhiên với các nội dung như ngành học, điểm chuẩn, tổ hợp môn, phương thức xét tuyển, học bổng. Khi đăng nhập, người dùng có thể lưu và xem lại lịch sử hội thoại theo từng phiên. Ngoài ra, hệ thống hỗ trợ lưu danh sách ngành quan tâm và cho phép người dùng gửi phản hồi về câu trả lời của chatbot dưới dạng hữu ích hoặc không hữu ích nhằm cải thiện chất lượng hệ thống. Bên cạnh đó, các tùy chọn nâng cao trải nghiệm như nhập liệu bằng giọng nói và nghe đọc nội dung bằng Text-to-Speech cũng được hỗ trợ tùy thuộc vào trình duyệt và quyền truy cập thiết bị của người dùng.

3.2. Yêu cầu hệ thống

3.2.1 Quản lý người dùng và phân quyền

Hệ thống lưu trữ thông tin tài khoản người dùng trong cơ sở dữ liệu SQLite với các trường cơ bản như tên đăng nhập, email, mật khẩu đã được mã hóa (hash) và các thông tin hồ sơ phục vụ cho việc tư vấn. Cơ chế phân quyền được thiết lập rõ ràng giữa hai vai trò Admin và User, đồng thời các API quan trọng được bảo vệ thông qua cơ chế xác thực bằng JSON Web Token (JWT) nhằm đảm bảo an toàn truy cập. Ngoài ra, hệ thống hỗ trợ chức năng khôi phục mật khẩu thông qua email bằng cách gửi token hoặc liên kết đặt lại mật khẩu, giúp người dùng có thể lấy lại quyền truy cập một cách an toàn và thuận tiện.

3.2.2 Quản lý dữ liệu tri thức tuyển sinh và Knowledge Graph

Dữ liệu nguồn được chuẩn hóa dưới dạng JSON (thư mục data/processed/) bao gồm các thông tin như ngành học, điểm chuẩn, tổ hợp môn, phương thức xét tuyển và học bổng nhằm đảm bảo tính đồng nhất và dễ dàng xử lý. Từ nguồn dữ liệu này, hệ thống thực hiện nạp hoặc rebuild vào cơ sở dữ liệu Neo4j dưới dạng các node và mối quan hệ, đồng thời có thể tích hợp embedding để phục vụ cho vector search khi cần thiết. Quá trình cập nhật dữ liệu luôn đảm bảo tính nhất quán và độ chính xác theo các văn bản chính thức của nhà trường, giúp hệ thống cung cấp thông tin tuyển sinh đáng tin cậy.

3.2.3 Tổ chức phiên hội thoại và xử lý câu hỏi

Hệ thống quản lý phiên trò chuyện (session chat) nhằm duy trì ngữ cảnh hội thoại giữa người dùng và chatbot, đồng thời hiển thị trạng thái đang xử lý khi chờ phản hồi từ backend hoặc mô hình LLM để nâng cao trải nghiệm người dùng. Toàn bộ tin nhắn giữa người dùng và chatbot được lưu trữ, cho phép theo dõi và truy xuất lại khi cần thiết; bên cạnh đó, hệ thống có thể cung cấp gợi ý câu hỏi liên quan và hiển thị nguồn thông tin tham chiếu nếu được trả về, giúp tăng tính minh bạch và hữu ích của câu trả lời.

3.2.4 Tra cứu và theo dõi tương tác người dùng

Hệ thống cho phép người dùng tra cứu và theo dõi các tương tác đã thực hiện thông qua việc xem lại lịch sử các phiên trò chuyện và toàn bộ nội dung đã trao đổi với chatbot. Bên cạnh đó, người dùng có thể quản lý danh sách ngành quan tâm, bao gồm việc lưu trữ, theo dõi và cập nhật các ngành học yêu thích, từ đó hỗ trợ quá trình tìm hiểu và ra quyết định phù hợp với nhu cầu cá nhân.

3.2.5 .Báo cáo và thống kê phục vụ quản trị

Hệ thống hỗ trợ các chức năng thống kê nhằm phục vụ quản trị và đánh giá hiệu quả hoạt động, bao gồm việc thống kê Knowledge Graph để theo dõi cấu trúc đồ thị như số lượng node, loại nhãn và các mối quan hệ. Ngoài ra, hệ thống có thể thống kê các nội dung tra cứu phổ biến hoặc các ngành học được quan tâm nhiều. Bên cạnh đó, hệ thống còn cho phép tổng hợp các phản hồi từ người dùng đối với câu trả lời của chatbot, từ đó hỗ trợ đánh giá chất lượng và cải thiện hiệu quả tư vấn trong tương lai.

3.2.6 Tích hợp AI (Gemini)

Hệ thống thực hiện phân loại ý định câu hỏi của người dùng nhằm xác định đúng nhu cầu thông tin cần truy vấn. Trong trường hợp cần thiết, hệ thống sử dụng kỹ thuật embedding kết hợp truy hồi thông tin và truy vấn cơ sở dữ liệu Neo4j để lấy ngữ cảnh liên quan từ Knowledge Graph. Dựa trên ngữ cảnh đã thu thập, mô hình Graph RAG được áp dụng để sinh câu trả lời phù hợp, đảm bảo tính chính xác và nhất quán. Ngoài ra, hệ thống còn có khả năng gợi ý các câu hỏi tiếp theo nhằm duy trì hội thoại liên tục và nâng cao trải nghiệm người dùng. Việc mô tả chức năng AI sinh đề trắc nghiệm không được đề cập do không thuộc phạm vi của đề tài.

3.3. Thiết kế kiến trúc hệ thống

3.3.1 Mục tiêu và nguyên tắc thiết kế

Hệ thống được thiết kế theo hướng tách tầng và tách biệt các thành phần nhằm đảm bảo kiến trúc rõ ràng và dễ mở rộng. Cụ thể, hệ thống phân tách rõ ràng giữa giao diện người dùng và phần xử lý nghiệp vụ cũng như truy cập dữ liệu, giúp nâng cao khả năng bảo trì và phát triển. Đồng thời, hệ thống sử dụng kết hợp hai loại cơ sở dữ liệu, trong đó SQLite đảm nhiệm lưu trữ dữ liệu liên quan đến người dùng và phiên hội thoại, còn Neo4j được sử dụng để lưu trữ tri thức tuyển sinh dưới dạng đồ thị phục vụ cho mô hình Graph RAG. Bên cạnh đó, hệ thống được thiết kế linh hoạt để dễ dàng triển khai trên các môi trường container như Docker và các nền tảng điện toán đám mây như Hugging Face Space thông qua việc cấu hình các biến môi trường cho kết nối Neo4j và API trí tuệ nhân tạo.

3.3.2 Kiến trúc tổng thể (Client – Server)

Hệ thống được xây dựng theo mô hình client–server trên nền tảng giao thức HTTP/HTTPS nhằm đảm bảo khả năng giao tiếp linh hoạt giữa các thành phần. Ở phía client (trình duyệt), hệ thống sử dụng React (Vite) kết hợp với Tailwind CSS để cung cấp giao diện người dùng, bao gồm các chức năng như chat tư vấn, xác thực tài khoản, xem lịch sử hội thoại, quản lý ngành quan tâm và gửi phản hồi, đồng thời có thể tích hợp khu vực quản trị dành cho admin nếu được triển khai. Ở phía server, hệ thống sử dụng FastAPI để cung cấp các REST API với tiền tố /api, thực hiện xác thực bằng JWT, điều phối luồng hội thoại và xử lý truy cập dữ liệu từ SQLite và Neo4j. Trong đó, SQLite được sử dụng để lưu trữ dữ liệu quan hệ phục vụ vận hành hệ thống như thông tin người dùng, phiên chat và tin nhắn, còn Neo4j được sử dụng để lưu trữ Knowledge Graph dưới dạng các node và mối quan hệ, đồng thời có thể tích hợp vector embedding nhằm phục vụ truy hồi ngữ cảnh. Ngoài ra, hệ thống tích hợp Gemini API đóng vai trò là mô hình ngôn ngữ lớn kết hợp với embedding để hỗ trợ sinh câu trả lời và nâng cao chất lượng xử lý ngôn ngữ tự nhiên.

3.3.3 Kiến trúc phân lớp (Layered Architecture)

Tầng Presentation (tầng giao diện) sử dụng công nghệ React và giao tiếp với backend thông qua REST API (ví dụ sử dụng axios) để thực hiện các yêu cầu dữ liệu. Tầng này đảm nhiệm việc hiển thị nội dung hội thoại dưới dạng markdown, quản lý trạng thái đang xử lý khi hệ thống đang trả lời, hỗ trợ chức năng đăng nhập và gửi kèm token JWT trong các request đến các API cần xác thực. Đồng thời, giao diện được thiết kế theo hướng responsive nhằm đảm bảo khả năng hiển thị tốt và trải nghiệm mượt mà trên nhiều loại thiết bị, đặc biệt là thiết bị di động.

Tầng Application/Business (tầng ứng dụng) được xây dựng bằng công nghệ FastAPI, đóng vai trò xử lý toàn bộ logic nghiệp vụ của hệ thống. Tầng này cung cấp các API chính như API chat để tiếp nhận câu hỏi từ người dùng và thực hiện pipeline Graph RAG bao gồm các bước phân loại ý định, truy xuất dữ liệu từ Neo4j, tổng hợp ngữ cảnh và sinh câu trả lời. Bên cạnh đó, hệ thống còn triển khai các API xác thực và quản lý hồ sơ người dùng như đăng ký, đăng nhập, xác thực bằng JWT và cập nhật thông tin cá nhân khi cần. Ngoài ra, các API dành cho quản trị viên được thiết kế với cơ chế giới hạn quyền, cho phép thực hiện các chức năng như thống kê Knowledge Graph, rebuild đồ thị và chỉnh sửa dữ liệu ngành trên Neo4j nhằm phục vụ công tác quản trị và vận hành hệ thống.

Tầng Data Access (tầng truy cập dữ liệu) chịu trách nhiệm làm việc trực tiếp với các hệ quản trị cơ sở dữ liệu của hệ thống. Đối với SQLite, hệ thống sử dụng SQLAlchemy theo mô hình ORM để thực hiện các thao tác với dữ liệu quan hệ, giúp tách biệt hoàn toàn phần truy vấn dữ liệu khỏi các route API và đảm bảo mã nguồn rõ ràng, dễ bảo trì. Đối với Neo4j, hệ thống sử dụng Neo4j Python Driver để thực thi các câu lệnh Cypher nhằm truy vấn và cập nhật dữ liệu đồ thị; các thao tác này thường được đóng gói trong các lớp hoặc hàm client riêng biệt để tái sử dụng, đồng thời giúp kiểm soát và tối ưu việc truy vấn dữ liệu trong toàn bộ hệ thống.

3.3.4 Luồng xử lý nghiệp vụ cốt lõi: Chat tư vấn (Graph RAG)

Hệ thống được thiết kế với luồng xử lý logic rõ ràng nhằm đảm bảo hiệu quả trong việc trả lời câu hỏi của người dùng. Khi người dùng gửi câu hỏi, request sẽ kèm theo session_id và token nếu đã đăng nhập. Backend tiếp nhận yêu cầu, thực hiện lưu tin nhắn của người dùng (theo thiết kế hệ thống), đồng thời tiến hành phân loại ý định để xác định chiến lược truy hồi phù hợp như tra cứu ngành học, điểm chuẩn hay học bổng. Tiếp theo, hệ thống truy xuất ngữ cảnh liên quan từ cơ sở dữ liệu Neo4j và có thể kết hợp vector search nếu pipeline có sử dụng embedding. Dữ liệu ngữ cảnh sau đó được tổng hợp thành prompt an toàn và gửi đến mô hình Gemini để sinh câu trả lời, đồng thời có thể kèm theo các gợi ý câu hỏi tiếp theo nhằm duy trì hội thoại. Cuối cùng, kết quả được trả về client dưới dạng JSON và tin nhắn phản hồi của chatbot được lưu lại vào SQLite để phục vụ việc quản lý lịch sử hội thoại.

3.3.5 Phân quyền và bảo mật kiến trúc

Hệ thống sử dụng JSON Web Token (JWT) để xác thực các API yêu cầu đăng nhập, trong đó client sẽ lưu trữ token và gửi kèm trong header Authorization khi thực hiện các request đến server. Cơ chế phân quyền admin được thiết lập nhằm đảm bảo các endpoint quản trị chỉ cho phép truy cập đối với các tài khoản có quyền phù hợp, thường là tài khoản quản trị viên. Đồng thời, hệ thống đảm bảo bảo vệ các thông tin nhạy cảm như thông tin kết nối Neo4j và khóa API của dịch vụ AI bằng cách chỉ cấu hình và lưu trữ trên server, không nhúng trực tiếp vào phía frontend, từ đó tăng cường tính bảo mật cho toàn bộ hệ thống.

3.3.6 Góc nhìn triển khai (Deployment view)

Frontend sau khi build sẽ được đóng gói dưới dạng static và được backend FastAPI phục vụ cùng với các API theo thiết kế triển khai (ví dụ thông qua Dockerfile). Ứng dụng được cấu hình lắng nghe trên cổng phù hợp với môi trường triển khai, chẳng hạn như cổng 7860 khi chạy trên nền tảng Hugging Face Space. Đối với cơ sở dữ liệu đồ thị, Neo4j có thể được triển khai dưới dạng một dịch vụ riêng sử dụng giao thức Bolt, và ứng dụng sẽ kết nối đến Neo4j thông qua các biến môi trường như NEO4J_URI, NEO4J_USER và NEO4J_PASSWORD nhằm đảm bảo tính linh hoạt và bảo mật trong quá trình triển khai.

3.4 Rủi ro

3.4.1 Rủi ro bảo mật và truy cập trái phép

Hệ thống có thể đối mặt với các rủi ro bảo mật như lộ JWT hoặc sử dụng mật khẩu yếu, rò rỉ thông tin cấu hình nhạy cảm như kết nối Neo4j hoặc API key của LLM trên phía client hoặc trong log, cũng như nguy cơ truy cập trái phép vào khu vực quản trị như quản lý Knowledge Graph hoặc thực hiện rebuild dữ liệu. Để giảm thiểu các rủi ro này, hệ thống áp dụng các biện pháp bảo mật như mã hóa mật khẩu dưới dạng hash, thiết lập phân quyền admin rõ ràng và kiểm tra token trên tất cả các endpoint nhạy cảm. Đồng thời, các thông tin bí mật chỉ được lưu trữ trên server thông qua biến môi trường, không đưa vào mã nguồn frontend hoặc lưu trong repository. Ngoài ra, khi triển khai công khai, hệ thống sử dụng giao thức HTTPS và cấu hình CORS hợp lý nhằm đảm bảo an toàn trong quá trình giao tiếp giữa các thành phần.

3.4.2 Rủi ro chất lượng và độ tin cậy của câu trả lời AI

Hệ thống có thể gặp rủi ro khi mô hình LLM sinh ra thông tin không chính xác (hallucination) hoặc diễn đạt sai lệch các chi tiết như điểm chuẩn hoặc mã ngành, đặc biệt khi ngữ cảnh truy xuất không đầy đủ hoặc dữ liệu nguồn chưa được cập nhật kịp thời. Để hạn chế vấn đề này, hệ thống ưu tiên sử dụng mô hình Graph RAG, trong đó câu trả lời được xây dựng dựa trên ngữ cảnh truy xuất từ Neo4j và có thể kèm theo nguồn tham chiếu nhằm tăng tính minh bạch. Đồng thời, dữ liệu nguồn dưới dạng JSON (data/processed/) được cập nhật thường xuyên theo các văn bản tuyển sinh chính thức, kèm theo quy trình rebuild lại đồ thị khi có thay đổi dữ liệu. Ngoài ra, trên giao diện hệ thống có thể bổ sung thông báo khuyến nghị người dùng đối chiếu thông tin với các thông báo tuyển sinh chính thức của nhà trường để đảm bảo độ chính xác.

3.4.3 Rủi ro dữ liệu tri thức (Neo4j) không đồng bộ hoặc seed lỗi

Hệ thống có thể gặp vấn đề khi dữ liệu đồ thị không đầy đủ, chẳng hạn như chỉ được seed một phần node, thiếu các mối quan hệ hoặc xảy ra tình trạng không đồng bộ giữa dữ liệu JSON và Neo4j sau khi cập nhật. Để khắc phục, cần xây dựng quy trình chuẩn hóa dữ liệu từ JSON trước khi thực hiện seed hoặc rebuild lại đồ thị, đồng thời thiết lập cơ chế kiểm tra số lượng node (ví dụ số lượng ngành) như một chỉ số "health" nhằm phát hiện sớm các sai lệch và thực hiện rebuild khi cần thiết. Bên cạnh đó, sau mỗi lần cập nhật dữ liệu lớn, hệ thống cần được kiểm thử lại thông qua một số câu hỏi mẫu liên quan đến ngành học, điểm chuẩn hoặc học bổng để đảm bảo tính chính xác và đầy đủ của dữ liệu.

3.4.4 Rủi ro hiệu năng và độ trễ (latency)

Hệ thống có nguy cơ bị giảm hiệu suất do quá trình gọi mô hình Gemini và truy vấn dữ liệu từ Neo4j có thể làm kéo dài thời gian phản hồi của chatbot; đồng thời các tác vụ xử lý nặng như rebuild đồ thị hoặc tạo embedding cũng có thể gây áp lực lên tài nguyên máy chủ. Để cải thiện, cần tối ưu hóa các câu truy vấn Cypher, giới hạn phạm vi tìm kiếm trong vector search (top-k) và hạn chế việc truy xuất dữ liệu đồ thị ở mức quá sâu khi không cần thiết. Ngoài ra, các tác vụ tốn tài nguyên nên được triển khai chạy nền để tránh ảnh hưởng đến hoạt động chính của hệ thống. Bên cạnh đó, việc thiết lập timeout hợp lý và cung cấp thông báo lỗi rõ ràng, thân thiện khi các dịch vụ bên ngoài phản hồi chậm cũng là giải pháp cần thiết nhằm nâng cao trải nghiệm người dùng.

3.4.5 Rủi ro phụ thuộc dịch vụ ngoài (Neo4j cloud, Gemini, email)

Trong quá trình vận hành, có thể phát sinh các vấn đề như mất kết nối đến Neo4j, hết quota API của LLM hoặc lỗi gửi email khôi phục mật khẩu do hạn chế SMTP trên môi trường cloud. Để xử lý, cần ghi log lỗi rõ ràng và kiểm tra trạng thái hệ thống thông qua các endpoint như /api/health (nếu có). Đồng thời, cấu hình dịch vụ gửi email phù hợp với môi trường triển khai (ví dụ Resend) và kiểm tra trước khi sử dụng. Ngoài ra, nên cung cấp thông báo fallback để người dùng nhận biết khi dịch vụ tạm thời gián đoạn.

3.4.6 Rủi ro quyền riêng tư và dữ liệu cá nhân

Việc lưu trữ lịch sử trò chuyện, thông tin hồ sơ như khối thi, điểm dự kiến và email nếu không được quản lý chặt chẽ có thể gây ra những lo ngại liên quan đến bảo mật dữ liệu cá nhân. Để giảm thiểu rủi ro này, hệ thống cần giới hạn việc thu thập dữ liệu ở mức cần thiết, thiết lập cơ chế phân quyền truy cập rõ ràng và đảm bảo không để lộ thông tin người dùng thông qua các API. Ngoài ra, có thể bổ sung chính sách xóa hoặc giới hạn thời gian lưu trữ dữ liệu tùy theo yêu cầu thực tế nhằm tăng cường bảo vệ quyền riêng tư của người dùng.

3.5. Mô tả tiến trình

3.5.1. Thu thập yêu cầu và khảo sát

Quá trình phân tích yêu cầu được thực hiện nhằm xác định rõ nhu cầu tra cứu thông tin tuyển sinh, bao gồm các nội dung như ngành học, điểm chuẩn, tổ hợp môn, phương thức xét tuyển và học bổng. Đồng thời, đề tài xác định đối tượng sử dụng chính là thí sinh và phụ huynh có nhu cầu tìm hiểu thông tin tuyển sinh, ngoài ra có thể bao gồm quản trị viên chịu trách nhiệm vận hành và quản lý cơ sở tri thức của hệ thống. Trên cơ sở đó, phạm vi chức năng của hệ thống được xác định và chuẩn hóa, bao gồm các chức năng chính như hệ thống chatbot tư vấn tuyển sinh, quản lý tài khoản người dùng và lưu trữ lịch sử hội thoại, quản lý danh sách ngành quan tâm, thu thập và xử lý phản hồi từ người dùng, cùng với các chức năng quản trị Knowledge Graph phục vụ công tác vận hành nếu được triển khai.

3.5.2. Phân tích và thiết kế

Trong giai đoạn thiết kế hệ thống, đề tài tiến hành xây dựng kiến trúc theo mô hình client–server kết hợp phân lớp rõ ràng gồm Presentation, API (Application) và Data Access nhằm đảm bảo tính tách biệt và dễ bảo trì. Đồng thời, hệ thống được thiết kế cơ sở dữ liệu gồm SQLite để lưu trữ dữ liệu quan hệ như người dùng, phiên chat, tin nhắn và Neo4j để biểu diễn tri thức tuyển sinh dưới dạng đồ thị với các node, mối quan hệ và có thể bao gồm embedding phục vụ truy hồi ngữ cảnh. Bên cạnh đó, luồng xử lý theo mô hình Graph RAG được thiết kế với các bước chính gồm phân loại ý định (intent), truy vấn dữ liệu từ Neo4j, xây dựng prompt và gửi đến Gemini để sinh câu trả lời.

Ngoài ra, đề tài cũng thực hiện xây dựng các sơ đồ phục vụ phân tích và trình bày trong báo cáo như Use Case, sơ đồ kiến trúc hệ thống, sequence xử lý chat, sơ đồ ERD cho SQLite và mô hình đồ thị cho Neo4j nhằm minh họa rõ cấu trúc và luồng hoạt động của hệ thống.

3.5.3. Chuẩn bị dữ liệu tri thức

Ở bước xử lý dữ liệu, đề tài tiến hành thu thập và tổng hợp thông tin từ các nguồn tuyển sinh chính thức, sau đó chuẩn hóa và lưu trữ dưới dạng file JSON trong thư mục data/processed/. Dựa trên nguồn dữ liệu này, hệ thống xây dựng và thực thi pipeline seed/rebuild để nạp dữ liệu vào Neo4j, đồng thời thiết lập các chỉ mục cần thiết như vector index (nếu có) nhằm phục vụ hiệu quả cho quá trình truy hồi ngữ cảnh trong mô hình Graph RAG.

3.5.4. Phát triển ChatBot

Đề tài triển khai backend sử dụng FastAPI nhằm cung cấp các API chức năng như chat tư vấn, xác thực người dùng bằng JWT và các chức năng quản trị như thống kê, rebuild và cập nhật dữ liệu ngành. Song song đó, frontend được phát triển bằng React để xây dựng giao diện chat, hỗ trợ xác thực, hiển thị lịch sử hội thoại, quản lý ngành quan tâm và đảm bảo khả năng hiển thị tốt trên thiết bị di động. Hệ thống đồng thời tích hợp mô hình Gemini để thực hiện các tác vụ như phân loại ý định, tạo embedding và sinh câu trả lời. Bên cạnh đó, việc kết nối và truy cập dữ liệu được thực hiện thông qua Neo4j Driver đối với cơ sở dữ liệu đồ thị và SQLAlchemy đối với SQLite, đảm bảo khả năng quản lý dữ liệu hiệu quả và phù hợp với kiến trúc của hệ thống.

3.5.5. Kiểm thử

Trong quá trình kiểm thử hệ thống, đề tài tiến hành đánh giá đầy đủ các thành phần nhằm đảm bảo hoạt động ổn định và đúng yêu cầu. Trước hết, các chức năng chính như đăng nhập, quá trình chat hoàn chỉnh (end-to-end), lưu trữ lịch sử hội thoại, quản lý danh sách ngành quan tâm và chức năng phản hồi được kiểm tra chi tiết. Bên cạnh đó, kiểm thử tích hợp được thực hiện để xác nhận khả năng kết nối với Neo4j, quá trình gọi mô hình LLM cũng như xử lý các tình huống như mất kết nối hoặc timeout. Đối với khu vực quản trị, hệ thống được kiểm tra các chức năng như rebuild dữ liệu và thống kê khi có triển khai. Ngoài ra, giao diện người dùng cũng được thử nghiệm trên cả máy tính và thiết bị di động nhằm đảm bảo khả năng hiển thị và trải nghiệm sử dụng nhất quán trên nhiều nền tảng.

3.5.6. Triển khai và vận hành

Quy trình triển khai bao gồm việc đóng gói ứng dụng bằng Dockerfile để đảm bảo môi trường chạy nhất quán và dễ dàng tái sử dụng. Tiếp theo, cấu hình đầy đủ các biến môi trường cần thiết như Neo4j, API key, URL và các thông số liên quan nhằm đảm bảo hệ thống hoạt động chính xác và bảo mật. Sau khi hoàn tất, tiến hành triển khai ứng dụng lên Hugging Face Space để đưa vào vận hành thực tế. Trong quá trình triển khai, cần theo dõi sát log build và runtime để kịp thời phát hiện và xử lý lỗi, đồng thời kiểm tra endpoint health

3.5.7. Bảo trì, cập nhật và mở rộng

Cần tiến hành cập nhật dữ liệu tuyển sinh theo từng mùa và kỳ để đảm bảo thông tin luôn chính xác và kịp thời, đồng thời thực hiện rebuild lại đồ thị dữ liệu nhằm cải thiện khả năng liên kết và truy xuất. Bên cạnh đó, dựa trên các phản hồi từ người dùng, hệ thống cần được sửa lỗi và tối ưu lại prompt cũng như cơ chế RAG để nâng cao hiệu suất hoạt động. Về hướng mở rộng trong tương lai, nên bổ sung thêm các intent nhằm hiểu rõ hơn nhu cầu người dùng, tích hợp đa dạng nguồn dữ liệu và tiếp tục cải thiện độ chính xác trong quá trình truy hồi thông tin.

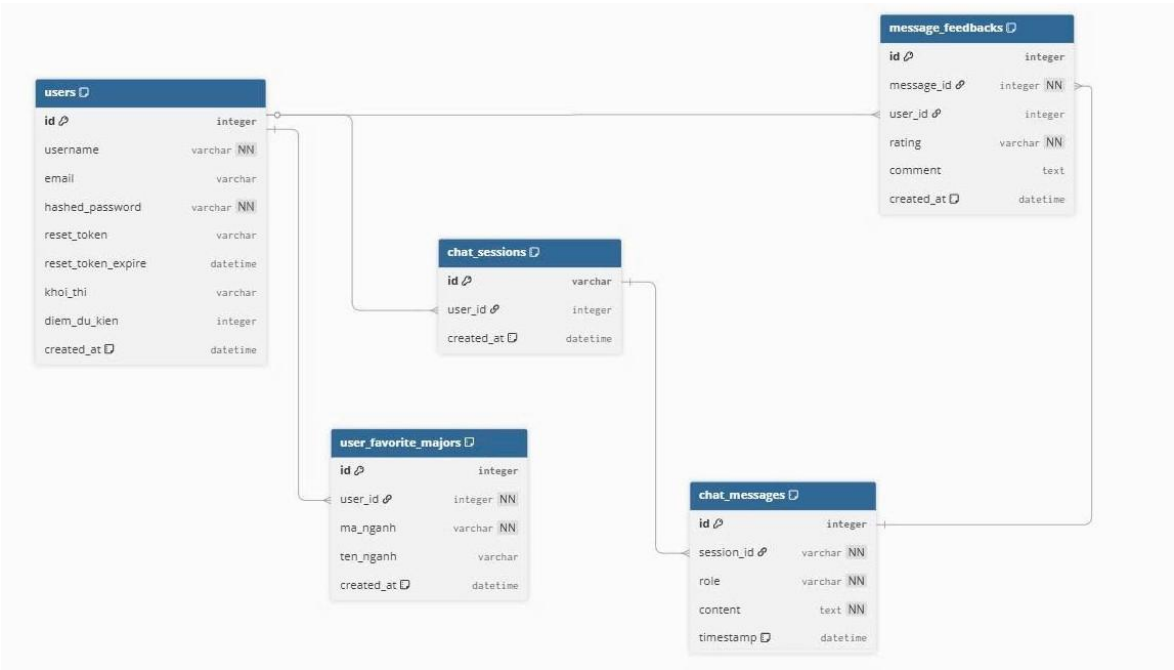
3.5.8. Theo dõi chất lượng và báo cáo kết quả

Theo dõi chất lượng: Người dùng có thể đánh giá câu trả lời (hữu ích / báo sai) theo từng tin nhắn bot; dữ liệu lưu trong SQLite để sau này rà soát nội dung và cải thiện tri thức/prompt. Lịch sử chat và ngành quan tâm giúp xem mức độ tương tác và nhu cầu tư vấn.

Báo cáo kết quả (Admin): Giao diện quản trị hiển thị thống kê Knowledge Graph (nút, quan hệ, nhãn), biểu đồ ngành được quan tâm, và lịch sử chat toàn hệ thống theo user → phiên, phục vụ giám sát vận hành và minh chứng hiệu quả trong phạm vi đồ án.

3.6 Mô hình thực thể quan hệ

3.6.1 Sơ đồ thực thể quan hệ



Hình 3.1 Sơ đồ ERD

3.6.2 Mô tả chi tiết các thực thể

Thực thể users lưu thông tin người dùng hệ thống, thuộc tính chính gồm: id (PK), username, email, hashed_password, reset_token, reset_token_expire, khoi_thi, diem_du_kien, created_at.

Thực thể chat_sessions lưu phiên trò chuyện của từng người dùng, thuộc tính chính gồm: id (PK), user_id (FK → users), created_at.

Thực thể chat_messages lưu các tin nhắn trong từng phiên chat, thuộc tính chính gồm: id (PK), session_id (FK → chat_sessions), role, content, timestamp.

Thực thể message_feedbacks lưu đánh giá/nhận xét của người dùng cho từng tin nhắn, thuộc tính chính gồm: id (PK), message_id (FK → chat_messages), user_id (FK → users), rating, comment, created_at.

Thực thể user_favorite_majors lưu danh sách ngành học yêu thích của người dùng, thuộc tính chính gồm: id (PK), user_id (FK → users), ma_nganh, ten_nganh, created_at.

3.6.3 Mô tả chi tiết các mối quan hệ

Một người dùng (users) có thể tạo nhiều phiên chat (chat_sessions), mỗi phiên chat thuộc về đúng một người dùng, với khóa ngoại chat_sessions.user_id → users.id.

Một phiên chat (chat_sessions) chứa nhiều tin nhắn (chat_messages), mỗi tin nhắn thuộc về đúng một phiên chat, với khóa ngoại chat_messages.session_id → chat_sessions.id.

Một tin nhắn (chat_messages) có thể nhận nhiều phản hồi/đánh giá (message_feedbacks), mỗi phản hồi thuộc về đúng một tin nhắn, với khóa ngoại message_feedbacks.message_id → chat_messages.id.

Một người dùng (users) có thể gửi nhiều phản hồi cho các tin nhắn (message_feedbacks), mỗi phản hồi được tạo bởi đúng một người dùng, với khóa ngoại message_feedbacks.user_id → users.id.

Một người dùng (users) có thể lưu nhiều ngành học yêu thích (user_favorite_majors), mỗi ngành yêu thích gắn với đúng một người dùng, với khóa ngoại user_favorite_majors.user_id → users.id.

3.7 Danh sách chức năng và luồng dữ liệu

Bảng 3.1 Danh sách chức năng và nguồn dữ liệu

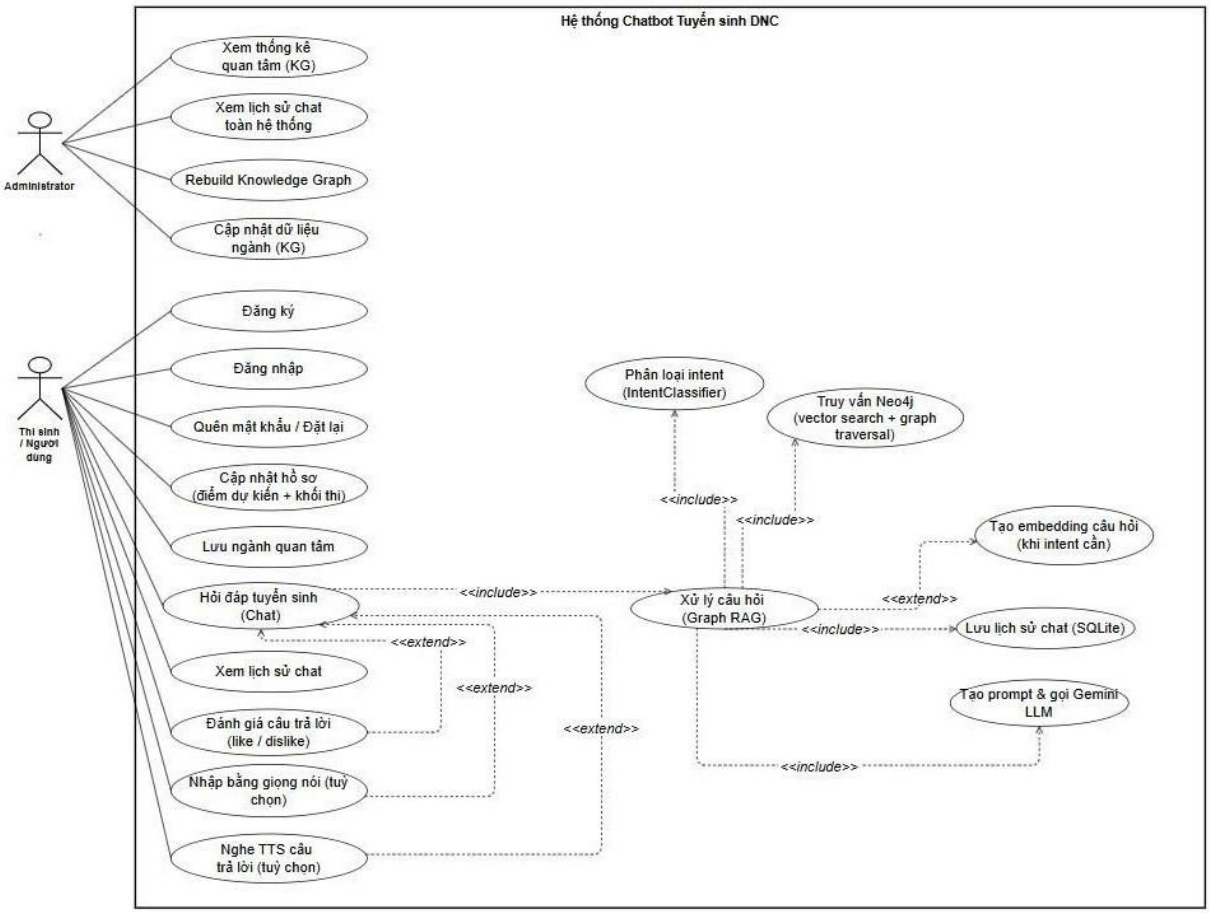
STT	Tác nhân	Chức năng (F1)	Chi tiết / Luồng (F2)	Input	Output / Xử lý	Bảng liên quan	Return
1	Khách / Người dùng	Chat tư vấn tuyển sinh	Nhập câu hỏi, nhận câu trả lời Markdown, có gợi ý câu hỏi tiếp theo	Tin nhắn của user	Sinh câu trả lời từ Graph RAG (Neo4j + Gemini)	chat_session, message	bot_message_id, markdown + gợi ý câu hỏi
2	Khách / Người dùng	Quản lý phiên chat	Gán session_id cho mỗi phiên trò chuyện	N/A	Tạo hoặc truy xuất phiên chat	chat_session	session_id
3	Khách / Người dùng	Phản hồi câu trả lời	Đánh giá hữu ích hoặc báo sai	bot_message_id, feedback	Lưu phản hồi vào cơ sở dữ liệu	feedback	success / fail
4	Khách / Người dùng	Tuỳ chọn UX	Nhập giọng nói, đọc nội dung (TTS, phụ thuộc trình duyệt)	Audio / Text	Chuyển đổi text ↔ speech	message	markdown / audio
5	Người dùng	Đăng ký / Đăng nhập / Đăng xuất	Sử dụng JWT	Email, mật khẩu	Tạo JWT, xác thực người dùng	user	JWT token / success

6	Người dùng	Cập nhật hồ sơ	Khối thi, điểm dự kiến để tư vấn cá nhân hóa	Dữ liệu hồ sơ	Cập nhật cơ sở dữ liệu	user_profile	success
7	Người dùng	Quên / đặt lại mật khẩu	Qua email (token / link)	Email	Gửi email reset, đặt mật khẩu mới	user	success / fail
8	Người dùng	Lịch sử hội thoại	Xem các phiên chat đã lưu	user_id	Truy xuất SQLite	chat_session, message	Danh sách chat
9	Người dùng	Ngành quan tâm	Thêm / xóa / xem danh sách ngành	Mã ngành	Lưu / xóa / truy xuất	favorites	Danh sách ngành
10	Admin	Thống kê Knowledge Graph	Xem số lượng node, label, quan hệ	N/A	Truy vấn Neo4j	Neo4j	Thống kê
11	Admin	Rebuild đồ thị	Nạp JSON → Neo4j, tạo constraint	JSON	Xóa / tạo node và quan hệ	Neo4j	success
12	Admin	Quản lý ngành / điểm	Thêm / sửa ngành hoặc điểm	Mã ngành, điểm	Cập nhật Neo4j	majors	success
13	Admin	Thống kê ngành phổ biến	Dựa trên search_count	N/A	Truy vấn Neo4j	majors	Thống kê

14	Admin	Xem lịch sử chat người dùng	Phục vụ vận hành	user_id	Truy xuất SQLite	chat_session, message	Danh sách chat
15	Hệ thống	Kiểm tra Neo4j	Tự seed / rebuild khi thiếu dữ liệu	N/A	Kiểm tra kết nối, seed dữ liệu	Neo4j	success / fail
16	Hệ thống	Triển khai	Docker / Hugging Face Space	N/A	Build frontend + FastAPI	Tất cả	Running
17	Client / User	Chat tư vấn (Graph RAG)	POST /api/chat	message, session_id, JWT	Phân loại intent → truy vấn Neo4j → sinh câu trả lời Gemini → lưu SQLite	chat_session, message	bot_message_id + markdown
18	User	Xác thực	JWT	Email, mật khẩu	Trả JWT	user	JWT token
19	User / Admin	Quên mật khẩu	Token email	Email	Tạo token → gửi email → đặt mật khẩu mới	user	success / fail
20	Admin	Tri thức offline / Rebuild	JSON → Neo4j → embedding	JSON chuẩn hóa	GraphBuilder tạo / cập nhật node, quan hệ, embedding	Neo4j	success

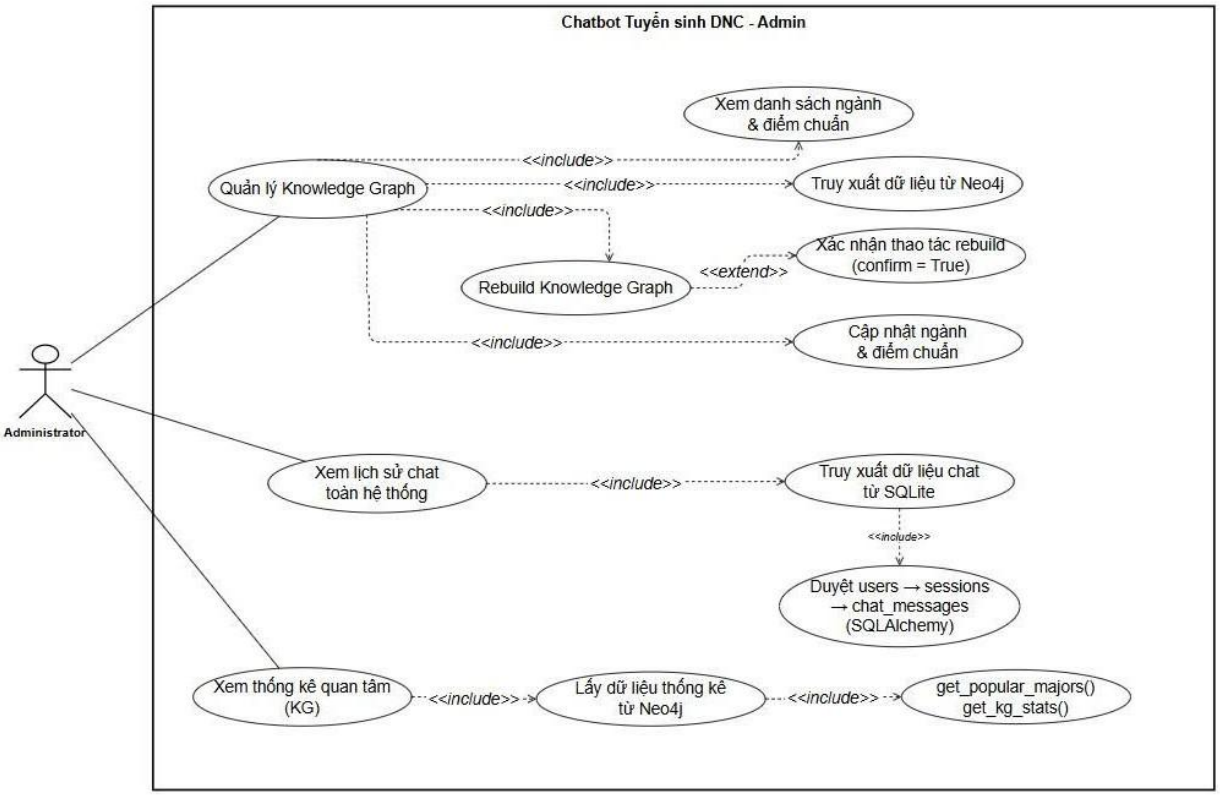
3.8 Mô hình Usecase

3.8.1 Sơ đồ Usecase tổng quát



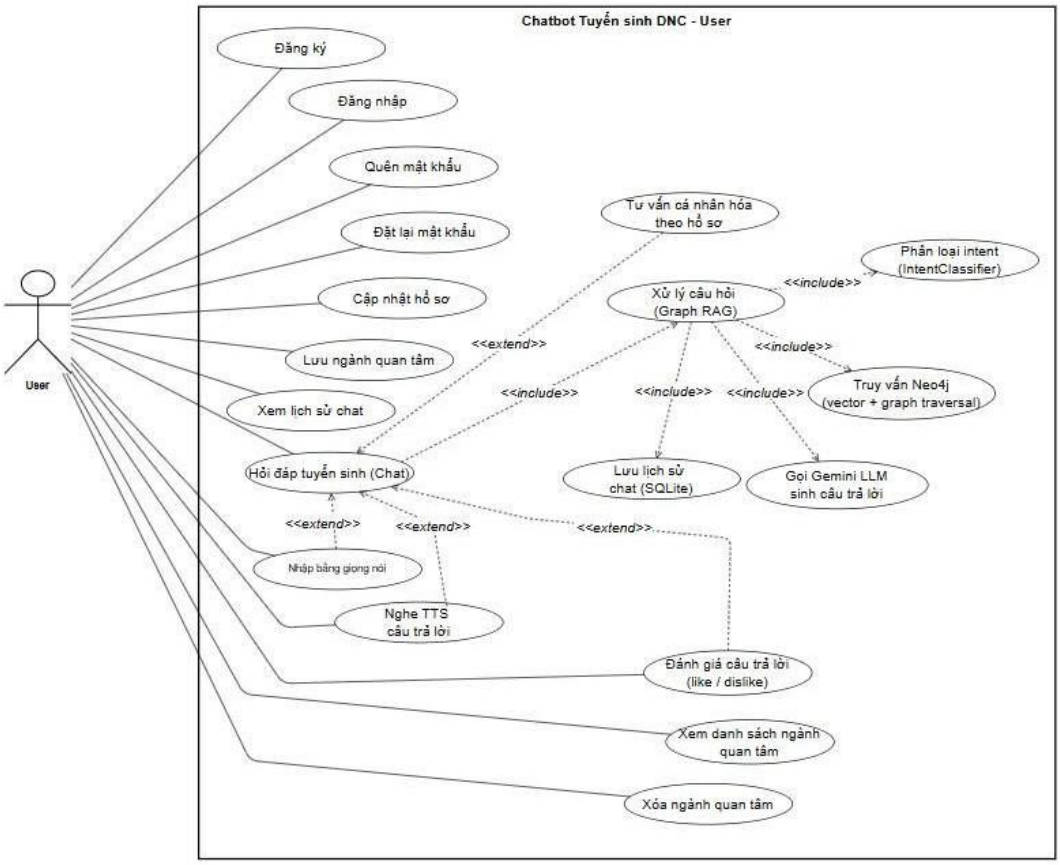
Hình 3.2 Sơ đồ Use Case tổng quát

3.8.2 Sơ đồ Usecase Admin



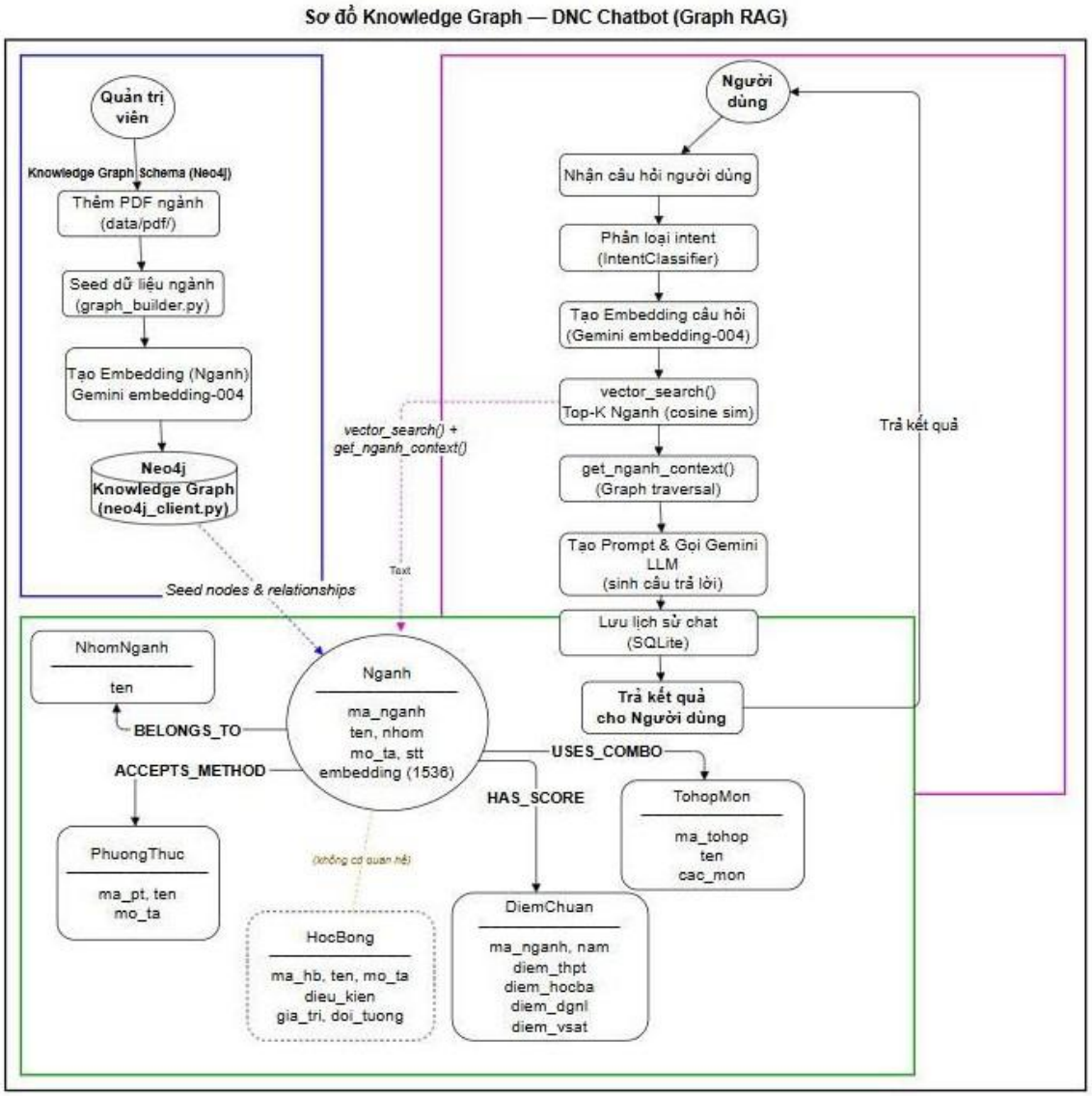
Hình 3.3 Sơ đồ Usecase Admin

3.8.3 Mô hình Usecase người dùng (User)



Hình 3.4 Sơ đồ usecase cho người dùng (User)

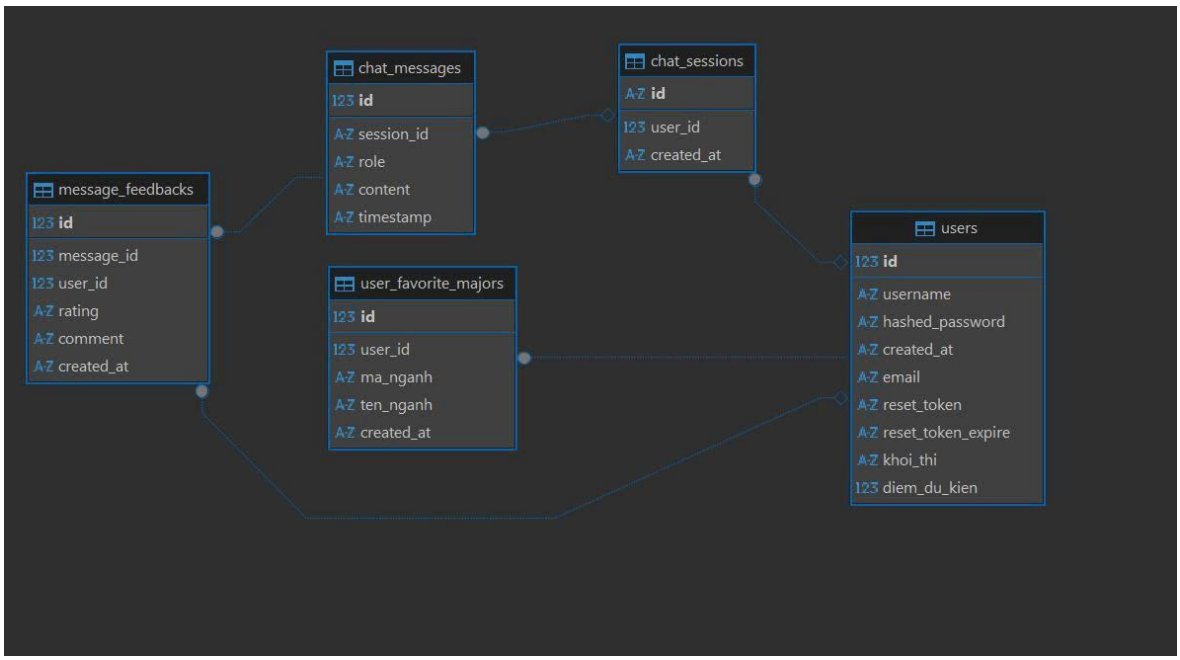
3.9 Sơ đồ kiến trúc Graph RAG và mô hình Knowledge Graph Neo4j



Hình 3.5 Sơ đồ kiến trúc Graph RAG và mô hình

CHƯƠNG 4: PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU

4.1 Mô hình cơ sở dữ liệu



Hình 4.1 Mô hình Database Diagrams

4.2 Các bảng cơ sở dữ liệu

Bảng 4.1 users(người dùng)

Thuộc tính	Kiểu dữ liệu	Khóa	Mô tả
id	INTEGER	PK, tự tăng	Khóa chính, định danh người dùng
username	VARCHAR	NOT NULL, UNIQUE	Tên đăng nhập, không trùng
email	VARCHAR	UNIQUE	Email người dùng, cho phép NULL
hashed_password	VARCHAR	NOT NULL	Mật khẩu đã hash
reset_token	VARCHAR	UNIQUE, NULL được	Token đặt lại mật khẩu
reset_token_expire	DATETIME	NULL được	Hạn dùng token reset
khoi_thi	VARCHAR	NULL được	Khối thi của người dùng
diem_du_kien	INTEGER	NULL được	Điểm dự kiến
created_at	DATETIME	-	Thời điểm tạo tài khoản

Bảng 4.2 chat_sessions(phiên bản hội thoại)

Thuộc tính	Kiểu dữ liệu	Khóa	Mô tả
id	VARCHAR(UID)	PK	UUID phiên chat
user_id	INTEGER	FK → users.id	Có thể NULL cho phiên khách
created_at	DATETIME	-	Thời điểm tạo phiên chat

Bảng 4.3 chat_messages (tin nhắn)

Thuộc tính	Kiểu dữ liệu	Khóa	Mô tả
id	INTEGER	PK, tự tăng	Khóa chính tin nhắn
session_id	VARCHAR	FK → chat_sessions.id, NOT NULL	Thuộc phiên chat nào
role	VARCHAR	NOT NULL	Vai trò gửi tin nhắn (user / bot)
content	TEXT	NOT NULL	Nội dung tin nhắn
timestamp	DATETIME	-	Thời điểm gửi tin nhắn

Bảng 4.4 user_favorite_majors(ngành yêu thích)

Thuộc tính	Kiểu dữ liệu	Khóa	Mô tả
id	INTEGER	PK, tự tăng	Khóa chính
user_id	INTEGER	FK → users.id, NOT NULL	Người dùng liên quan
ma_nganh	VARCHAR	NOT NULL	Mã ngành
ten_nganh	VARCHAR	NULL được	Tên ngành
created_at	DATETIME	-	Thời điểm lưu

Bảng 4.5 message_feedbacks

Thuộc tính	Kiểu dữ liệu	Khóa / Ràng buộc	Mô tả
id	INTEGER	PK, tự tăng	Mã phản hồi duy nhất
message_id	INTEGER	FK → chat_messages.id, NOT NULL	Tin nhắn được phản hồi
user_id	INTEGER	FK → users.id, NULL được	Người phản hồi (NULL nếu ẩn danh)
rating	VARCHAR	NOT NULL	Đánh giá tin nhắn (up / down)
comment	TEXT	NULL được	Nội dung phản hồi
created_at	DATETIME	-	Thời điểm tạo phản hồi

4.3 Ràng buộc toàn vẹn

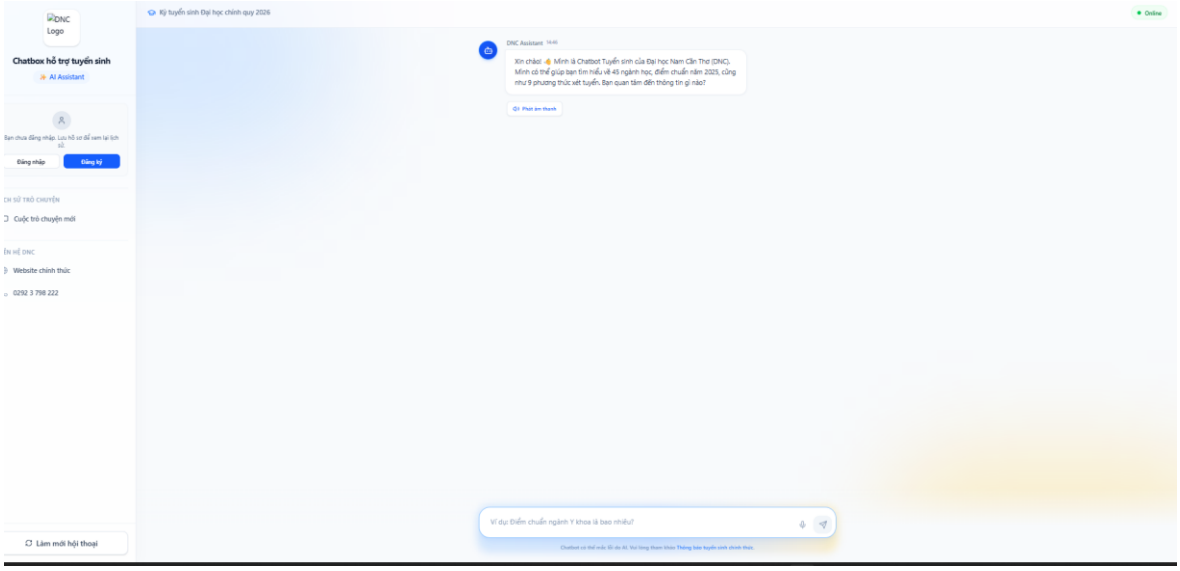
1. chat_sessions.user_id → users.id
2. chat_messages.session_id → chat_sessions.id
3. user_favorite_majors.user_id → users.id
4. message_feedbacks.message_id → chat_messages.id
5. message_feedbacks.user_id → users.id (NULL được, phản hồi ẩn danh)

CHƯƠNG 5: THIẾT KẾ GIAO DIỆN

5.1. Giao diện dành cho người dùng (USER)

5.1.1 Giao diện màn hình chính người dùng

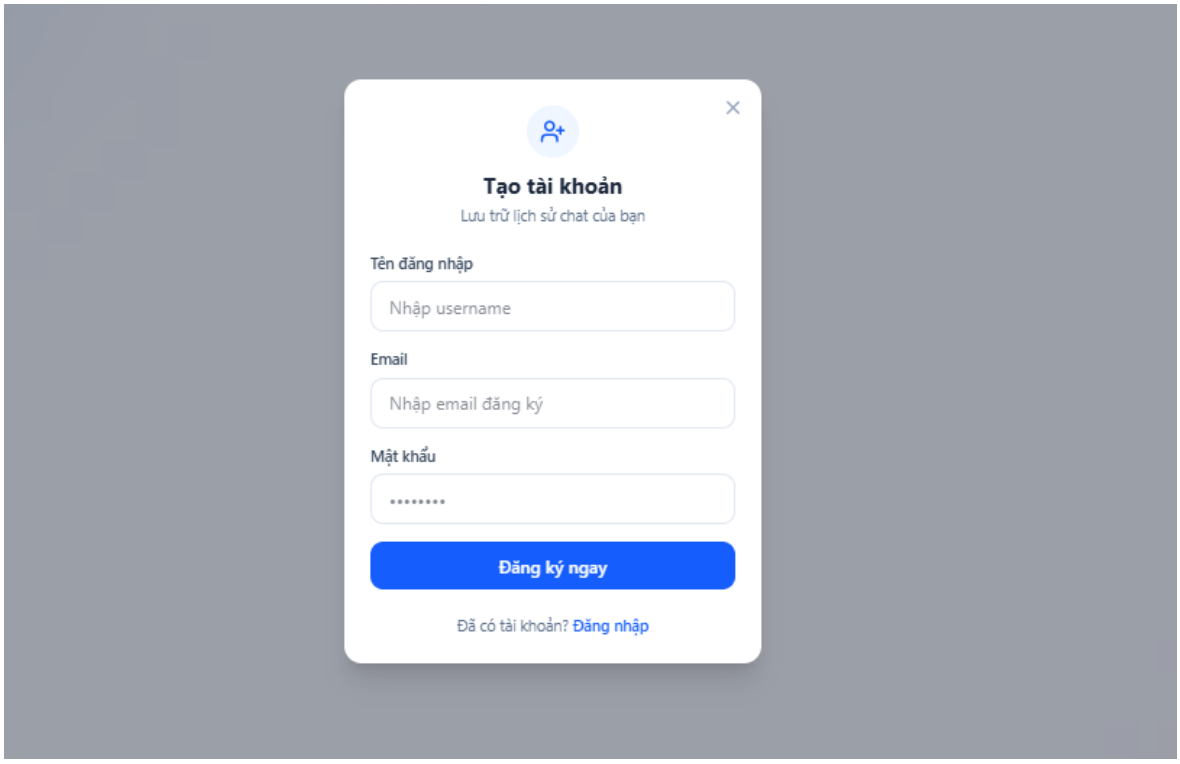
Người dùng có thể sử dụng chatbot để tra cứu thông tin tuyển sinh một cách nhanh chóng và tiện lợi như tìm hiểu ngành học, điểm chuẩn, phương thức xét tuyển hoặc đặt các câu hỏi liên quan đến trường. Ngoài ra, người dùng còn có thể nhập câu hỏi trực tiếp hoặc sử dụng giọng nói để tương tác, giúp quá trình tìm kiếm thông tin trở nên dễ dàng và tiết kiệm thời gian hơn.



Hình 5 1 Giao diện màn hình chính người dùng

5.1.2 Giao diện đăng ký

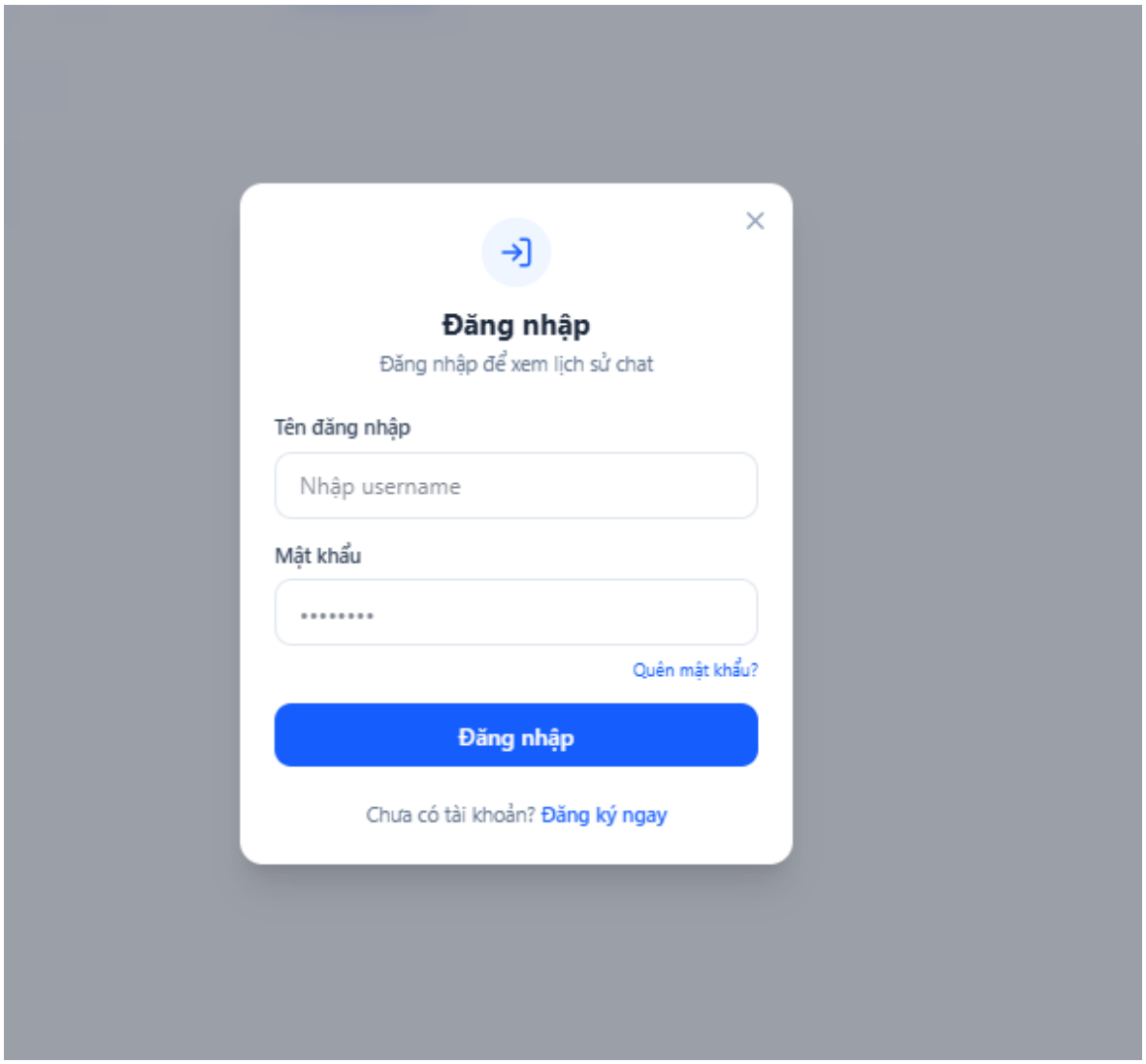
Giao diện đăng cho phép người dùng đăng ký để lưu lại lịch sử trò chuyện và sử dụng đầy đủ các chức năng. Người dùng cần nhập tên đăng nhập, email và mật khẩu, sau đó nhấn "Đăng ký ngay" để hoàn tất. Ngoài ra, hệ thống cũng hỗ trợ chuyển sang đăng nhập nếu đã có tài khoản, giúp thao tác nhanh chóng và thuận tiện.



Hình 5.2 Giao diện đăng ký

5.1.3 Giao diện Đăng Nhập.

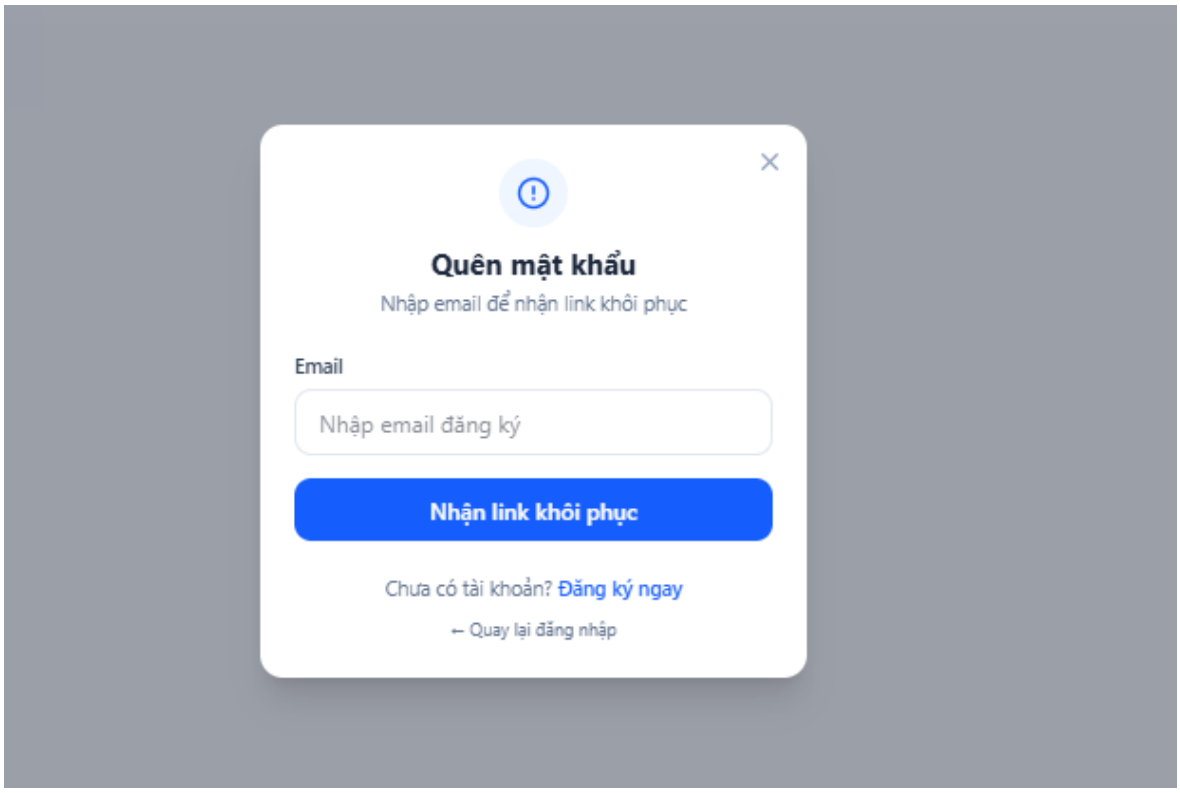
Giao diện đăng nhập của chatbot, cho phép người dùng truy cập vào hệ thống để xem lại lịch sử trò chuyện và sử dụng đầy đủ các tính năng. Người dùng chỉ cần nhập tên đăng nhập và mật khẩu, sau đó nhấn "Đăng nhập" để tiếp tục



Hình 5.3 Giao diện đăng nhập

5.1.4 Giao quên mật khẩu

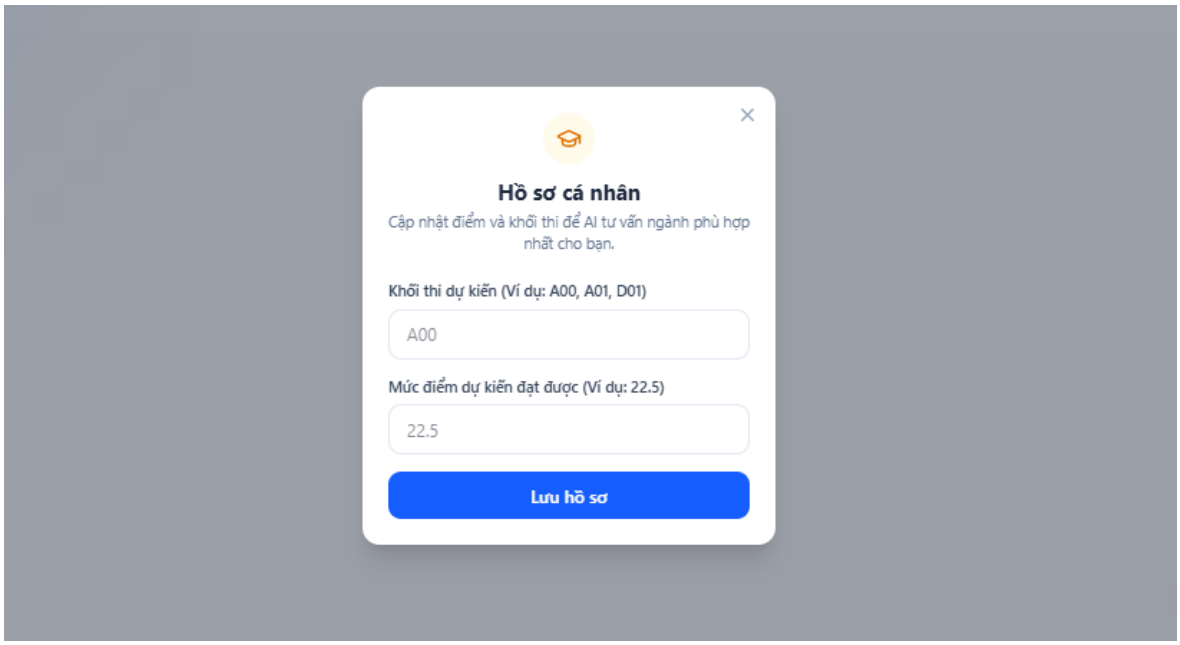
Giao diện quên mật khẩu của chatbot, hỗ trợ người dùng khôi phục tài khoản khi không nhớ mật khẩu. Người dùng chỉ cần nhập email đã đăng ký, sau đó nhấn "Nhận link khôi phục" để nhận hướng dẫn đặt lại mật khẩu.



Hình 5.4 Giao diện quên mật khẩu

5.1.5 Giao diện hồ sơ cá nhân

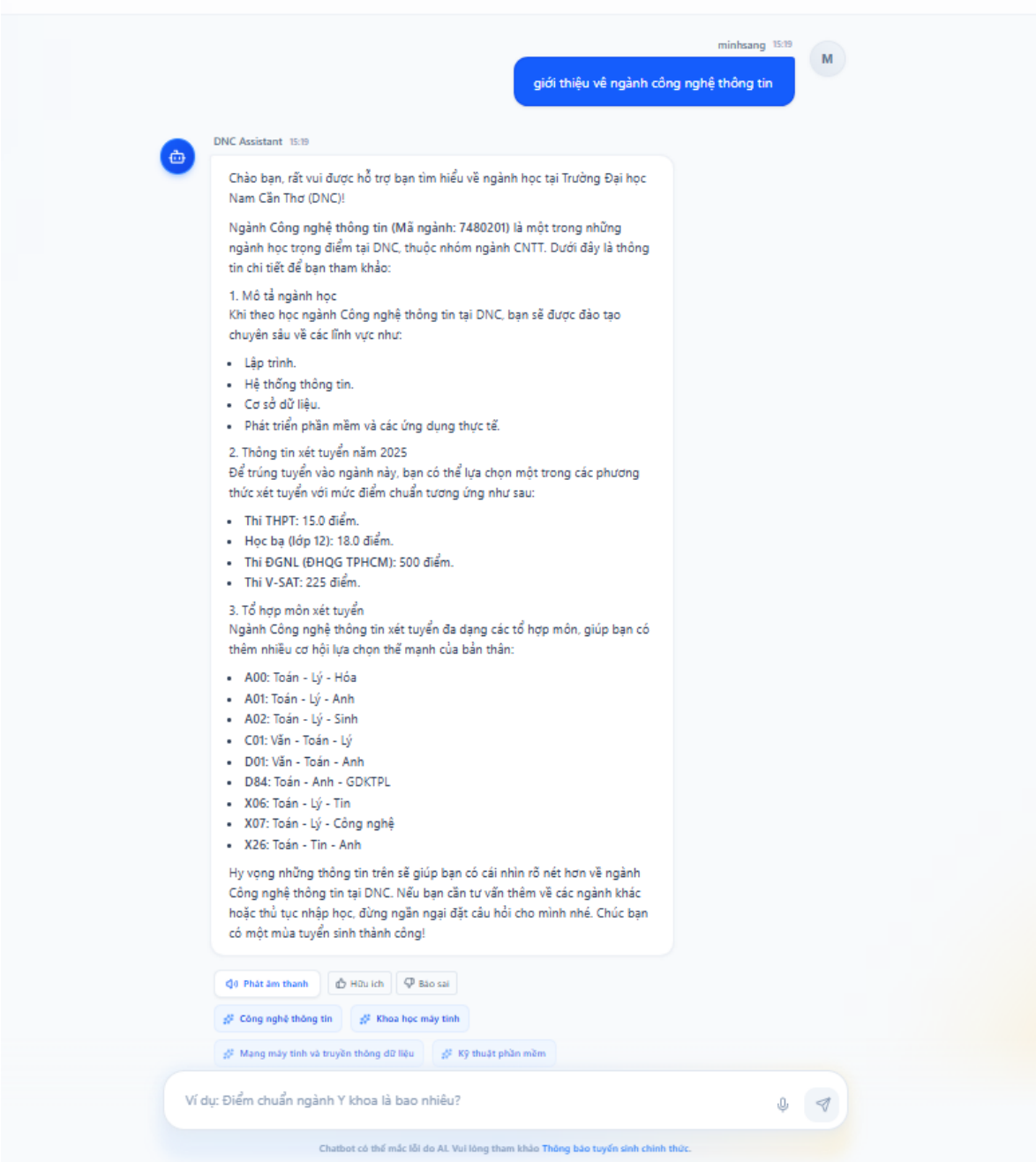
Đây là giao diện hồ sơ cá nhân của chatbot, cho phép người dùng nhập thông tin học tập để hệ thống đưa ra tư vấn phù hợp. Người dùng có thể điền khối thi dự kiến (như A00, A01, D01) và mức điểm mong muốn đạt được. Dựa trên các thông tin này, chatbot sẽ gợi ý ngành học phù hợp nhất. Sau khi hoàn tất, người dùng nhấn "Lưu hồ sơ" để lưu lại dữ liệu và sử dụng cho các lần tư vấn tiếp theo.



Hình 5.5 Giao diện hồ sơ cá nhân

5.1.6 Giao diện khi người dùng đặt câu hỏi

Giao diện chatbot DNC Assistant được thiết kế trực quan, gồm ba phần chính: tin nhắn người dùng nổi bật bên phải (màu xanh), phản hồi chi tiết của bot bên trái kèm các nút tiện ích/gợi ý, và thanh nhập liệu đa năng ở dưới cùng.

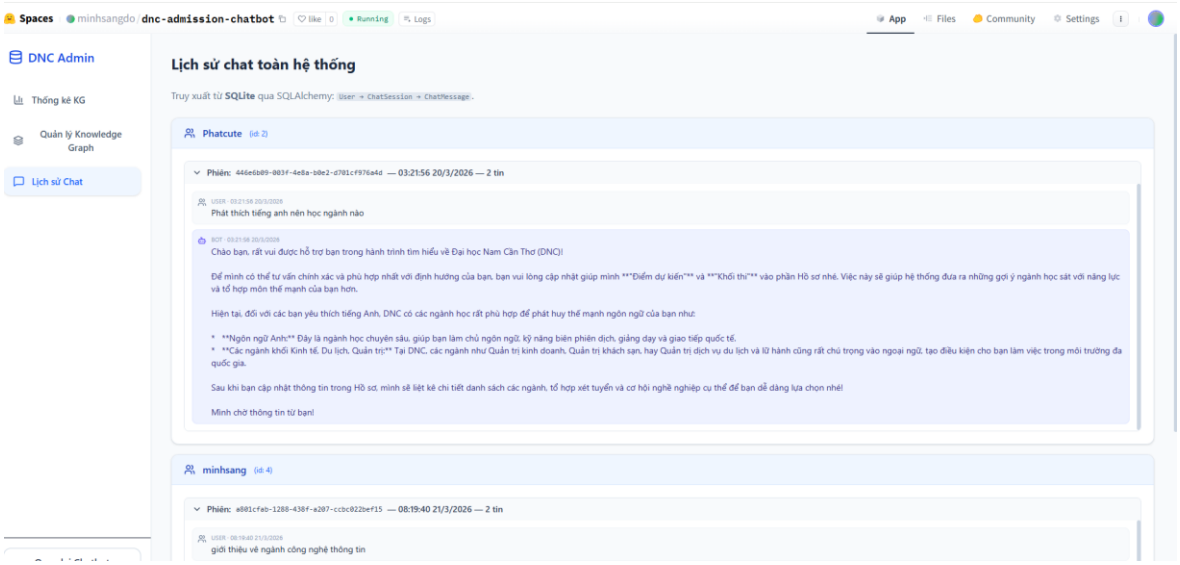


Hình 5.6 Giao diện khi người dùng đặt câu hỏi.

5.2. Giao diện dành cho quản trị viên (ADMIN)

5.2.1 Giao diện lịch sử chat

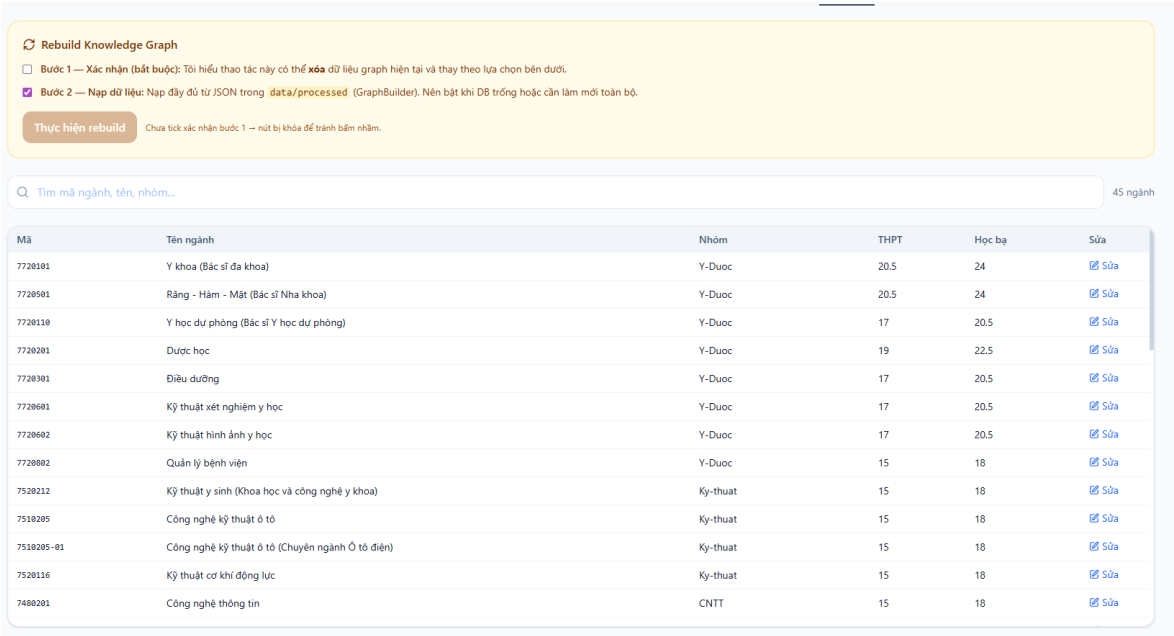
Giao diện quản trị (Admin) cho phép quản trị viên theo dõi lịch sử chat toàn hệ thống: dữ liệu được nhóm theo người dùng, trong mỗi user có các phiên hội thoại (kèm mã phiên, thời gian, số tin nhắn); khi mở rộng có thể xem toàn bộ đoạn hội thoại giữa người dùng và bot.



Hình 5.7 Giao diện lịch sử chat

5.2.2 Giao diện quản lý Knowledge Graph

Giao diện quản trị tri thức cho phép admin rebuild Knowledge Graph từ dữ liệu JSON trong data/processed (có bước xác nhận và nút thực hiện), đồng thời xem danh sách ngành kèm điểm THPT / học bạ, tìm kiếm theo mã–tên–nhóm, và chỉnh sửa từng ngành qua nút Sửa để cập nhật thông tin tuyển sinh hiển thị cho chatbot.

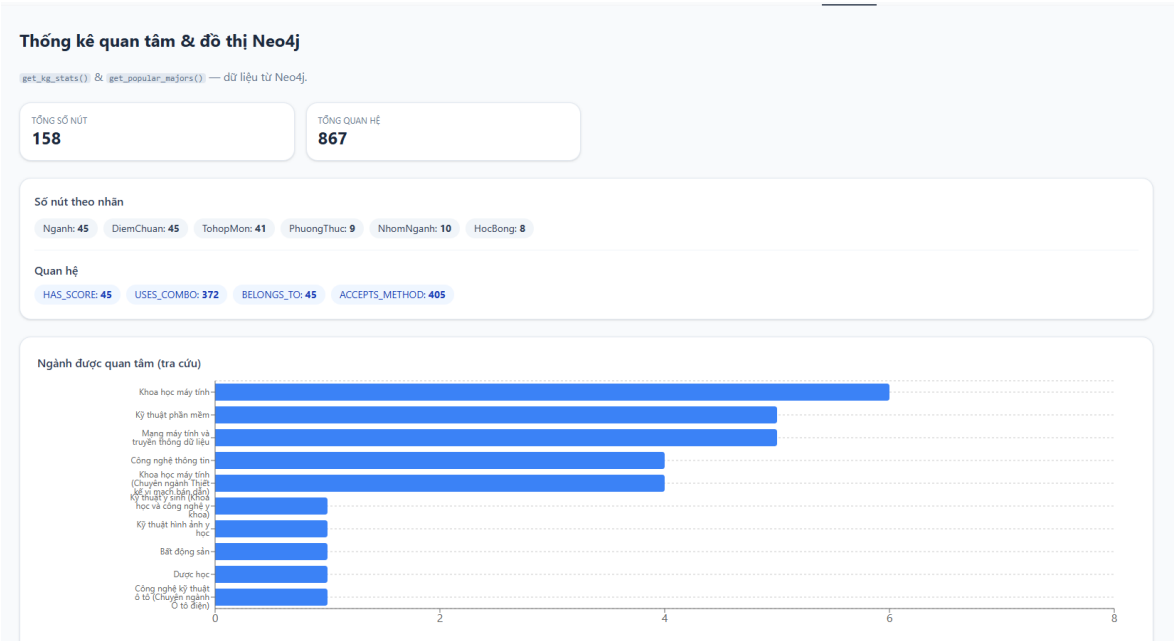


Mã	Tên ngành	Nhóm	THPT	Học bạ	Sửa
7720101	Y khoa (Bác sĩ đa khoa)	Y-Duoc	20.5	24	Sửa
7720501	Răng - Hàm - Mặt (Bác sĩ Nha khoa)	Y-Duoc	20.5	24	Sửa
7720110	Y học dự phòng (Bác sĩ Y học dự phòng)	Y-Duoc	17	20.5	Sửa
7720201	Dược học	Y-Duoc	19	22.5	Sửa
7720301	Điều dưỡng	Y-Duoc	17	20.5	Sửa
7720601	Kỹ thuật xét nghiệm y học	Y-Duoc	17	20.5	Sửa
7720602	Kỹ thuật hình ảnh y học	Y-Duoc	17	20.5	Sửa
7720802	Quản lý bệnh viện	Y-Duoc	15	18	Sửa
7520212	Kỹ thuật y sinh (Khoa học và công nghệ y khoa)	Ky-thuat	15	18	Sửa
7510205	Công nghệ kỹ thuật ô tô	Ky-thuat	15	18	Sửa
7510205-01	Công nghệ kỹ thuật ô tô (Chuyên ngành Ô tô điện)	Ky-thuat	15	18	Sửa
7520116	Kỹ thuật cơ khí động lực	Ky-thuat	15	18	Sửa
7480201	Công nghệ thông tin	CNTT	15	18	Sửa

Hình 5.8 Giao diện quản lý người dùng

5.2.3 Giao diện thống kê quan tâm & đồ thị Neo4j

Cho phép quản trị viên xem nhanh quy mô của Knowledge Graph, bao gồm tổng số nút và tổng số quan hệ, đồng thời hiển thị phân bố theo nhãn: Nganh, DiemChuan, TohopMon, PhuongThuc, NhomNganh, HocBong. Hệ thống cũng thống kê số lượng từng loại quan hệ: HAS_SCORE, USES_COMBO, BELONGS_TO, ACCEPTS_METHOD, và cung cấp biểu đồ cột ngang theo dõi các ngành được tra cứu nhiều nhất, nhằm đánh giá mức độ quan tâm của người dùng.



Hình 5.9 Giao diện thống kê quan tâm & đồ thị Neo4j

CHƯƠNG 6: THỬ NGHIỆM VÀ ĐÁNH GIÁ

6.1 Kiểm thử

6.1.1 Chức năng đăng nhập

Bảng 6.1 Kiểm thử chức năng đăng nhập

STT	Tên chức năng	Mô tả	Dữ liệu	Các bước	Kết quả mong đợi
1	Kiểm tra form rỗng	Bấm đăng nhập khi chưa nhập đủ dữ liệu	Trống	Để trống → Đăng nhập	Không đăng nhập, báo lỗi trường bắt buộc
2	Kiểm tra sai tài khoản/mật khẩu	Nhập sai thông tin	Sai	Nhập sai → Đăng nhập	Không đăng nhập, báo lỗi server
3	Kiểm tra đăng nhập đúng	Đăng nhập hợp lệ	Đúng	Nhập đúng → Đăng nhập	Đăng nhập thành công, lưu token

6.1.2 Chức năng quên mật khẩu

Bảng 6.2 Kiểm thử quên mật khẩu

STT	Tên chức năng	Mô tả	Dữ liệu	Các bước	Kết quả mong đợi
1	Email rỗng	Chưa nhập email	Trống	Bấm gửi	Báo thiếu email
2	Email không tồn tại	Email chưa đăng ký	Sai email	Gửi	Thông báo chung (không lộ có/không tài khoản)
3	Gửi link	Email đúng	Email đã đăng ký	Gửi	Nhận email có link/token
4	Token lỗi/hết hạn	Token không hợp lệ	Token sai/hết hạn	Đổi MK	Báo lỗi token
5	Đổi MK thành công	Token đúng	Token hợp lệ + MK mới	Xác nhận	Đổi MK OK, đăng nhập lại được

6.1.3 Chức năng đăng ký

Bảng 6.3 Kiểm thử chức năng đăng ký

STT	Tên chức năng	Mô tả	Dữ liệu	Các bước	Kết quả mong đợi
1	Thiếu trường bắt buộc	Bấm đăng ký khi thiếu thông tin	Thiếu	Điền thiếu → Đăng ký	Không tạo tài khoản
2	Trùng username/email	Đăng ký trùng dữ liệu	Trùng	Đăng ký	Báo đã tồn tại
3	Đăng ký thành công	Tạo tài khoản mới	Hợp lệ	Đăng ký	Tạo thành công

6.1.4 Chức năng chat

Bảng 6.4 Kiểm thử chức năng chat

STT	Tên chức năng	Mô tả	Dữ liệu	Các bước	Kết quả mong đợi
1	Câu hỏi rỗng	Gửi không có nội dung	Rỗng	Gửi	Không gửi / báo lỗi
2	Hỏi điểm/ngành	Hỏi nội dung tuyển sinh	Có nội dung	Gửi	Bot trả lời
3	Hỏi học bổng	Hỏi học bổng	Có nội dung	Gửi	Bot trả lời theo nhánh học bổng

6.1.5 Chức năng phản hồi

Bảng 6.5 Kiểm thử chức năng phản hồi

STT	Tên chức năng	Mô tả	Dữ liệu	Các bước	Kết quả mong đợi
1	Gửi phản hồi	Đánh giá câu trả lời bot	up/down	Bấm phản hồi	Thông báo thành công

6.1.6 Chức năng ngành thêm ngành yêu thích

Bảng 6.6 Kiểm thử ngành yêu thích

STT	Tên chức năng	Mô tả	Dữ liệu	Các bước	Kết quả mong đợi
1	Thêm ngành hợp lệ	Lưu mã ngành	Mã hợp lệ	Lưu	Thêm vào danh sách
2	Thêm trùng	Lưu lại cùng mã	Trùng mã	Lưu	Báo lỗi trùng
3	Xóa ngành	Xóa khỏi danh sách	Mã đã có	Xóa	Xóa thành công

6.1.7 Chức năng phân quyền

Bảng 6.7 Phân quyền

STT	Tên chức năng	Mô tả	Dữ liệu	Các bước	Kết quả mong đợi
1	User thường	Vào admin	Token user	Mở admin	Bị chặn
2	Tài khoản admin	Vào admin	Token admin	Mở admin	Vào được

6.1.8 Chức năng chức năng xây dựng lại đồ thị tri thức Neo4j

Bảng 6.8 Kiểm thử chức năng xây dựng lại đồ thị tri thức Neo4j

STT	Tên chức năng	Mô tả	Dữ liệu	Các bước	Kết quả mong đợi
1	Chưa tick xác nhận	Bấm rebuild	Không confirm	Bấm rebuild	Không thực hiện / nhắc xác nhận
2	Rebuild đầy đủ	Tick confirm + full ingest	confirm=true , JSON đủ	Thực hiện rebuild	Nạp lại đồ thị

6.1.9 Chức năng sửa lưu ngành và điểm

Bảng 6.9 Kiểm thử chức năng sửa lưu ngành và điểm

STT	Mục kiểm thử	Thao tác	Kết quả mong đợi
1	Lưu hợp lệ	Chọn một ngành, sửa điểm chuẩn/THPT… → Lưu	Thông báo thành công, dữ liệu cập nhật trên bảng
2	Lỗi (tuỳ chọn)	Nhập giá trị không hợp lệ hoặc mất mạng khi lưu	Hiển thị lỗi rõ ràng, không crash trang

6.1.10 Chức năng tìm kiếm ngành trong bảng

Bảng 6.10 Kiểm thử chức năng tìm kiếm ngành trong bảng

STT	Mục kiểm thử	Thao tác	Kết quả mong đợi
1	Lọc theo từ khóa	Tab Quản lý Knowledge Graph, nhập từ khóa vào ô tìm kiếm	Danh sách chỉ còn các ngành khớp (mã/tên); xóa ô tìm → hiện lại đủ

6.2 Đánh giá

Đồ án đã xây dựng thành công một hệ thống tư vấn tuyển sinh theo mô hình client–server hiện đại, trong đó giao diện React (Vite) giao tiếp với FastAPI, kết hợp SQLite để quản lý dữ liệu người dùng và Neo4j để lưu trữ tri thức dạng đồ thị. Việc tích hợp Gemini API theo hướng Graph RAG giúp nâng cao chất lượng câu trả lời nhờ truy xuất ngữ cảnh có cấu trúc, thay vì chỉ sinh nội dung chung chung. Hệ thống đã đáp ứng được các chức năng cơ bản cho người dùng như đăng ký, đăng nhập, trò chuyện với chatbot, lưu lịch sử, đánh dấu ngành quan tâm và gửi phản hồi. Đồng thời, hệ thống cũng cung cấp khu vực quản trị với các tính năng thống kê, quản lý dữ liệu ngành, theo dõi lịch sử chat và vận hành pipeline tri thức. Ngoài ra, sản phẩm đã có bản demo trực tuyến và tài liệu README tương đối đầy đủ, hỗ trợ việc trình bày và tái hiện hệ thống.

Bên cạnh các kết quả đạt được, đồ án vẫn còn một số hạn chế như việc kiểm thử chưa được chuẩn hóa đầy đủ, chủ yếu dừng ở mức kiểm tra thủ công và chưa có tài liệu kiểm thử chi tiết. Cơ chế bảo mật và phân quyền còn đơn giản, chưa hỗ trợ mô hình vai trò đa cấp hoặc hệ thống ghi log đầy đủ. Hệ thống còn phụ thuộc vào dịch vụ bên thứ ba (Gemini API) mà chưa có phương án dự phòng rõ ràng khi gặp sự cố hoặc giới hạn sử dụng. Dữ liệu tri thức chưa đảm bảo cập nhật hoàn toàn theo thực tế nếu không có quy trình duy trì định kỳ. Ngoài ra, chưa có đánh giá toàn diện về hiệu năng, khả năng mở rộng khi có nhiều người dùng đồng thời, cũng như chưa tối ưu trải nghiệm đa nền tảng và khả năng tiếp cận (accessibility), do đó hệ thống hiện vẫn phù hợp với phạm vi đồ án hơn là triển khai ở quy mô thực tế.

CHƯƠNG 7: KẾT LUẬN

Trong quá trình nghiên cứu và triển khai đồ án cá nhân, em đã tự xây dựng hệ thống chatbot tư vấn tuyển sinh đại học cho Trường Đại học Nam Cần Thơ (DNC) với các chức năng trọng tâm như: hỏi–đáp thông minh dựa trên tri thức có cấu trúc, đăng ký/đăng nhập và quản lý phiên người dùng, lưu lịch sử trò chuyện, đánh dấu ngành học quan tâm, phản hồi chất lượng câu trả lời, cùng khu vực quản trị dành cho tài khoản admin để theo dõi thống kê Knowledge Graph, ngành được tra cứu nhiều, quản lý và cập nhật thông tin ngành–điểm trên đồ thị, khởi tạo lại/tái xây dựng KG và xem lịch sử chat toàn hệ thống theo người dùng và phiên.

Ứng dụng được em phát triển theo mô hình web hiện đại (React + Vite, giao diện thân thiện) kết hợp API FastAPI (Python) và cơ sở dữ liệu quan hệ SQLite cho dữ liệu người dùng–phiên–tin nhắn, đồng thời sử dụng Neo4j để biểu diễn và truy vấn Knowledge Graph phục vụ Graph RAG. Hệ thống có bản demo trực tuyến (Hugging Face Space), thuận tiện minh chứng và trình bày kết quả.

Việc ứng dụng chatbot vào công tác tư vấn tuyển sinh giúp chuẩn hóa kênh cung cấp thông tin, giảm phụ thuộc vào tra cứu rời rạc, và hướng người học đến câu trả lời có căn cứ từ dữ liệu ngành–điểm–phương thức được lưu trong đồ thị. Dữ liệu hội thoại và hành vi người dùng được lưu trữ tập trung, hỗ trợ theo dõi, thống kê và quản trị, từ đó nâng cao hiệu quả tư vấn và cải tiến nội dung tri thức theo thời gian.

Một điểm nổi bật của đồ án là tích hợp AI (Gemini API) vào pipeline: vừa dùng embedding và mô hình ngôn ngữ để hiểu câu hỏi và sinh câu trả lời, vừa kết hợp truy xuất ngữ cảnh từ Neo4j nhằm tăng độ chính xác, liên kết và khả năng giải thích theo ngữ cảnh ngành học tại DNC. Cách tiếp cận Graph RAG góp phần cải thiện trải nghiệm so với trả lời thuần "mô hình ngôn ngữ không neo dữ liệu", đồng thời phù hợp với bối cảnh tuyển sinh có nhiều thực thể và quan hệ (ngành, điểm, tổ hợp, phương thức…).

Bên cạnh đó, em đã phân tách người dùng thường và quản trị viên (đăng nhập admin) để kiểm soát các thao tác nhạy cảm như xem lịch sử toàn hệ thống, cấu hình dữ liệu KG và thao tác rebuild, đảm bảo an toàn vận hành trong phạm vi đồ án. Các màn hình thống kê và báo cáo trong admin cũng hỗ trợ theo dõi mức độ sử dụng, xu hướng quan tâm ngành và tình trạng dữ liệu đồ thị, làm cơ sở điều chỉnh nội dung và vận hành.

Tổng thể, hệ thống em xây dựng đáp ứng tốt yêu cầu nghiệp vụ đặt ra trong phạm vi đồ án (tư vấn tuyển sinh có tri thức đồ thị và giao diện quản trị cơ bản), đồng thời mở hướng phát triển như: tăng cường phân quyền đa vai trò và nhật ký thao tác, tối ưu hiệu năng và chi phí khi tải cao, mở rộng kiểm thử đa thiết bị, và hoàn thiện quy trình cập nhật dữ liệu tuyển sinh chính thức để sẵn sàng triển khai thực tế lâu dài.

TÀI LIỆU THAM KHẢO

[1] Trương Hùng Chen (2020), Giáo trình Phân tích và thiết kế hệ thống thông tin, Khoa Công Nghệ Thông Tin, Trường Đại Học Nam Cần Thơ.

[2] Ths. Phan Thị Xuân Trang (2019), Giáo trình Cơ sở dữ liệu, Khoa Công Nghệ Thông Tin, Trường Đại Học Nam Cần Thơ.

[3] Ths. Phan Thị Xuân Trang (2019), Giáo trình Hệ quản trị cơ sở dữ liệu, Khoa Công Nghệ Thông Tin, Trường Đại Học Nam Cần Thơ.

[4] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS), 33.

[5] Edge, D., Trinh, H., Cheng, R., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv preprint arXiv:2404.16130.

[6] Neo4j, Inc. (n.d.). Neo4j Documentation (phiên bản 5.x).https://neo4j.com/docs/

[7] Ramírez, S. (n.d.). FastAPI Documentation.https://fastapi.tiangolo.com/

[8] Meta Open Source. (n.d.). React Documentation. Truy cập tại: https://react.dev/

[9] Google. (n.d.). Gemini API Documentation (Google AI for Developers). https://ai.google.dev/gemini-api/docs

[10] Hipp, D., Kennedy, D., & developers. (n.d.). SQLite Documentation. https://www.sqlite.org/docs.html